

Chapter 3. Processes

The concept of a process is fundamental to any multiprogramming operating system. A process is usually defined as an instance of a program in execution; thus, if 16 users are running vi at once, there are 16 separate processes (although they can share the same executable code). Processes are often called tasks or threads in the Linux source code.

In this chapter, we discuss static properties of processes and then describe how process switching is performed by the kernel. The last two sections describe how processes can be created and destroyed. We also describe how Linux supports multithreaded applications as mentioned in [Chapter 1](#), it relies on so-called lightweight processes (LWP).

3.1. Processes, Lightweight Processes, and Threads

The term "process" is often used with several different meanings. In this book, we stick to the usual OS textbook definition: a process is an instance of a program in execution. You might think of it as the collection of data structures that fully describes how far the execution of the program has progressed.

Processes are like human beings: they are generated, they have a more or less significant life, they optionally generate one or more child processes, and eventually they die. A small difference is that sex is not really common among processes each process has just one parent.

From the kernel's point of view, the purpose of a process is to act as an entity to which system resources (CPU time, memory, etc.) are allocated.

When a process is created, it is almost identical to its parent. It receives a (logical) copy of the parent's address space and executes the same code as the parent, beginning at the next instruction following the process creation system call. Although the parent and child may share the pages containing the program code (text), they have separate copies of the data (stack and heap), so that changes by the child to a memory location are invisible to the parent (and vice versa).

While earlier Unix kernels employed this simple model, modern Unix systems do not. They support multithreaded applications user programs having many relatively independent execution flows sharing a large portion of the application data structures. In such systems, a process is composed of several user threads (or simply threads), each of which represents an execution flow of the process. Nowadays, most multithreaded applications are written using standard sets of library functions called pthread (POSIX thread) libraries .

Older versions of the Linux kernel offered no support for multithreaded applications. From the kernel point of view, a multithreaded application was just a normal process. The multiple execution flows of a multithreaded application were created, handled, and scheduled entirely in User Mode, usually by means of a POSIX-compliant pthread library.

However, such an implementation of multithreaded applications is not very satisfactory. For instance, suppose a chess program uses two threads: one of them controls the graphical chessboard, waiting for the moves of the human player and showing the moves of the computer, while the other thread ponders the next move of the game. While the first thread waits for the human move, the second thread should run continuously, thus exploiting the thinking time of the human player. However, if the chess program is just a single process, the first thread cannot simply issue a blocking system call waiting for a user action; otherwise, the second thread is blocked as well. Instead, the first thread must employ sophisticated nonblocking techniques to ensure that the process remains runnable.

Linux uses lightweight processes to offer better support for multithreaded applications. Basically, two lightweight processes may share some resources, like the address space, the open files, and so on. Whenever one of them modifies a shared resource, the other immediately sees the change. Of course, the two processes must synchronize themselves when accessing the shared resource.

A straightforward way to implement multithreaded applications is to associate a lightweight process with each thread. In this way, the threads can access the same set of application data structures by simply sharing the same memory address space, the same set of open files, and so on; at the same time, each thread can be scheduled independently by the kernel so that one may sleep while another remains runnable. Examples of POSIX-compliant pthread libraries that use Linux's lightweight processes are LinuxThreads, Native POSIX Thread Library (NPTL), and IBM's Next Generation Posix Threading Package (NGPT).

POSIX-compliant multithreaded applications are best handled by kernels that support "thread groups ." In Linux a thread group is basically a set of lightweight processes that implement a multithreaded application and act as a whole with regards to some system calls such as `getpid( )` , `kill( )` , and `_exit( )` . We are going to describe them at length later in this chapter.

3.2. Process Descriptor

To manage processes, the kernel must have a clear picture of what each process is doing. It must know, for instance, the process's priority, whether it is running on a CPU or blocked on an event, what address space has been assigned to it, which files it is allowed to address, and so on. This is the role of the process descriptor a `task_struct` type structure whose fields contain all the information related to a single process.^[*] As the repository of so much information, the process descriptor is rather complex. In addition to a large number of fields containing process attributes, the process descriptor contains several pointers to other data structures that, in turn, contain pointers to other structures. Figure 3-1 describes the Linux process descriptor schematically.

[*] The kernel also defines the `task_t` data type to be equivalent to `struct task_struct`.

The six data structures on the right side of the figure refer to specific resources owned by the process. Most of these resources will be covered in future chapters. This chapter focuses on two types of fields that refer to the process state and to process parent/child relationships.

3.2.1. Process State

As its name implies, the `state` field of the process descriptor describes what is currently happening to the process. It consists of an array of flags, each of which describes a possible process state. In the current Linux version, these states are mutually exclusive, and hence exactly one flag of `state` always is set; the remaining flags are cleared. The following are the possible process states:

TASK_RUNNING

The process is either executing on a CPU or waiting to be executed.

TASK_INTERRUPTIBLE

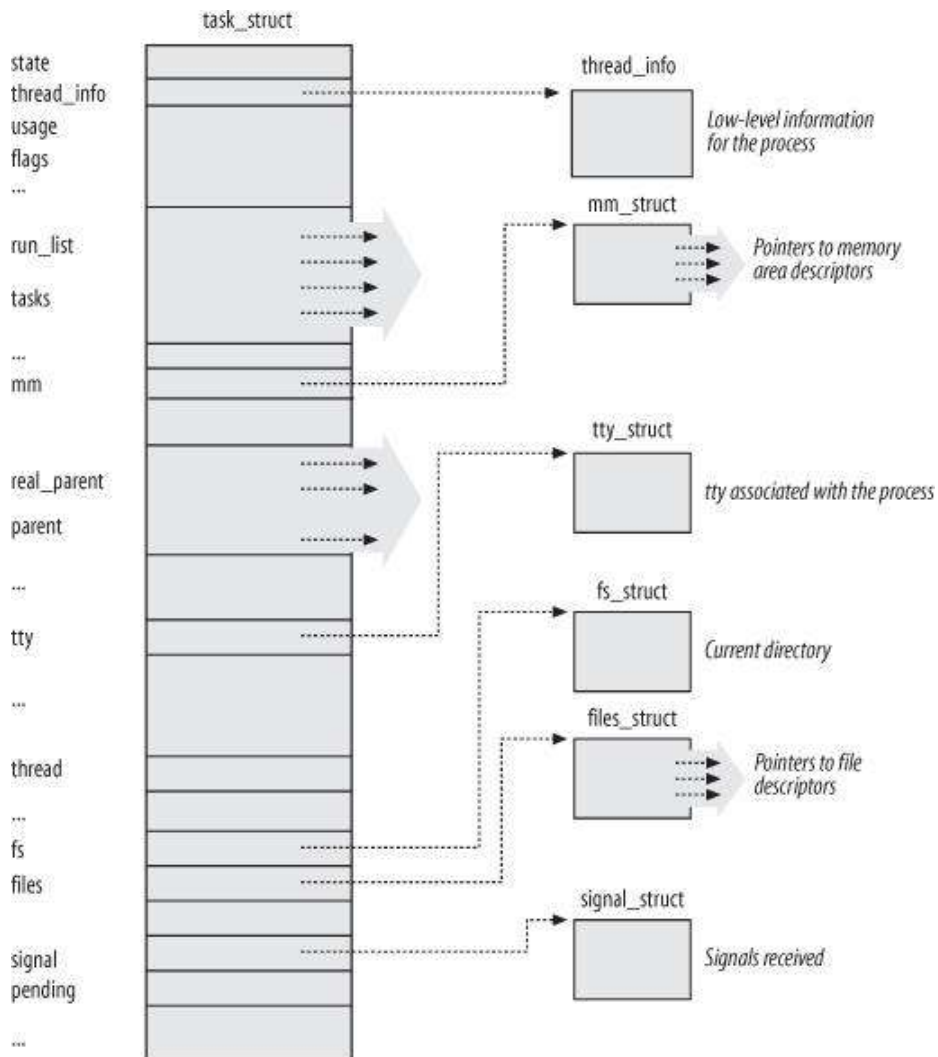
The process is suspended (sleeping) until some condition becomes true. Raising a hardware interrupt, releasing a system resource the process is waiting for, or delivering a signal are examples of conditions that might wake up the process (put its state back to `TASK_RUNNING`).

TASK_UNINTERRUPTIBLE

Like `TASK_INTERRUPTIBLE`, except that delivering a signal to the sleeping process leaves its state unchanged. This process state is seldom used. It is valuable, however, under certain specific conditions in which a process must wait until a given event occurs without being interrupted. For instance, this state may

be used when a process opens a device file and the corresponding device driver starts probing for a corresponding hardware device. The device driver must not be interrupted until the probing is complete, or the hardware device could be left in an unpredictable state.

Figure 3-1. The Linux process descriptor



TASK_STOPPED

Process execution has been stopped; the process enters this state after receiving a `SIGSTOP`, `SIGTSTP`, `SIGTTIN`, or `SIGTTOU` signal.

TASK_TRACED

Process execution has been stopped by a debugger. When a process is being monitored by another (such as when a debugger executes a `ptrace()` system call to monitor a test program), each signal may put the process in the `TASK_TRACED` state.

Two additional states of the process can be stored both in the `state` field and in the `exit_state` field of the process descriptor; as the field name suggests, a process

reaches one of these two states only when its execution is terminated:

EXIT_ZOMBIE

Process execution is terminated, but the parent process has not yet issued a `wait4( )` or `waitpid( )` system call to return information about the dead process.^[*] Before the `wait( )`-like call is issued, the kernel cannot discard the data contained in the dead process descriptor because the parent might need it. (See the section "[Process Removal](#)" near the end of this chapter.)

[*] There are other `wait( )`-like library functions, such as `wait3( )` and `wait( )`, but in Linux they are implemented by means of the `wait4( )` and `waitpid( )` system calls.

EXIT_DEAD

The final state: the process is being removed by the system because the parent process has just issued a `wait4( )` or `waitpid( )` system call for it. Changing its state from `EXIT_ZOMBIE` to `EXIT_DEAD` avoids race conditions due to other threads of execution that execute `wait( )`-like calls on the same process (see [Chapter 5](#)).

The value of the `state` field is usually set with a simple assignment. For instance:

```
p->state = TASK_RUNNING;
```

The kernel also uses the `set_task_state` and `set_current_state` macros: they set the state of a specified process and of the process currently executed, respectively. Moreover, these macros ensure that the assignment operation is not mixed with other instructions by the compiler or the CPU control unit. Mixing the instruction order may sometimes lead to catastrophic results (see [Chapter 5](#)).

3.2.2. Identifying a Process

As a general rule, each execution context that can be independently scheduled must have its own process descriptor; therefore, even lightweight processes, which share a large portion of their kernel data structures, have their own `task_struct` structures.

The strict one-to-one correspondence between the process and process descriptor makes the 32-bit address^[†] of the `task_struct` structure a useful means for the kernel to identify processes. These addresses are referred to as process descriptor pointers. Most of the references to processes that the kernel makes are through process descriptor pointers.

[†] As already noted in the section "[Segmentation in Linux](#)" in [Chapter 2](#), although technically these 32 bits are only the offset component of a logical address, they coincide with the linear address.

On the other hand, Unix-like operating systems allow users to identify processes by means of a number called the Process ID (or PID), which is stored in the `pid` field of the process descriptor. PIDs are numbered sequentially: the PID of a newly created process is normally the PID of the previously created process increased by one. Of course, there is an upper limit on the PID values; when the kernel reaches such limit, it must start recycling the lower, unused PIDs. By default, the maximum PID number is 32,767 (`PID_MAX_DEFAULT - 1`); the system administrator may reduce this limit by writing a smaller value into the `/proc /sys/kernel/pid_max` file (`/proc` is the mount point of a special filesystem, see the section "[Special Filesystems](#)" in [Chapter 12](#)). In 64-bit architectures, the system administrator can enlarge the maximum PID number up to 4,194,303.

When recycling PID numbers, the kernel must manage a `pidmap_array` bitmap that denotes which are the PIDs currently assigned and which are the free ones. Because a page frame contains 32,768 bits, in 32-bit architectures the `pidmap_array` bitmap is stored in a single page. In 64-bit architectures, however, additional pages can be added to the bitmap when the kernel assigns a PID number too large for the current bitmap size. These pages are never released.

Linux associates a different PID with each process or lightweight process in the system. (As we shall see later in this chapter, there is a tiny exception on multiprocessor systems.) This approach allows the maximum flexibility, because every execution context in the system can be uniquely identified.

On the other hand, Unix programmers expect threads in the same group to have a common PID. For instance, it should be possible to send a signal specifying a PID that affects all threads in the group. In fact, the POSIX 1003.1c standard states that all threads of a multithreaded application must have the same PID.

To comply with this standard, Linux makes use of thread groups. The identifier shared by the threads is the PID of the thread group leader, that is, the PID of the first lightweight process in the group; it is stored in the `tgid` field of the process descriptors. The `getpid( )` system call returns the value of `tgid` relative to the current process instead of the value of `pid`, so all the threads of a multithreaded application share the same identifier. Most processes belong to a thread group consisting of a single member; as thread group leaders, they have the `tgid` field equal to the `pid` field, thus the `getpid( )` system call works as usual for this kind of process.

Later, we'll show you how it is possible to derive a true process descriptor pointer efficiently from its respective PID. Efficiency is important because many system calls such as `kill( )` use the PID to denote the affected process.

3.2.2.1. Process descriptors handling

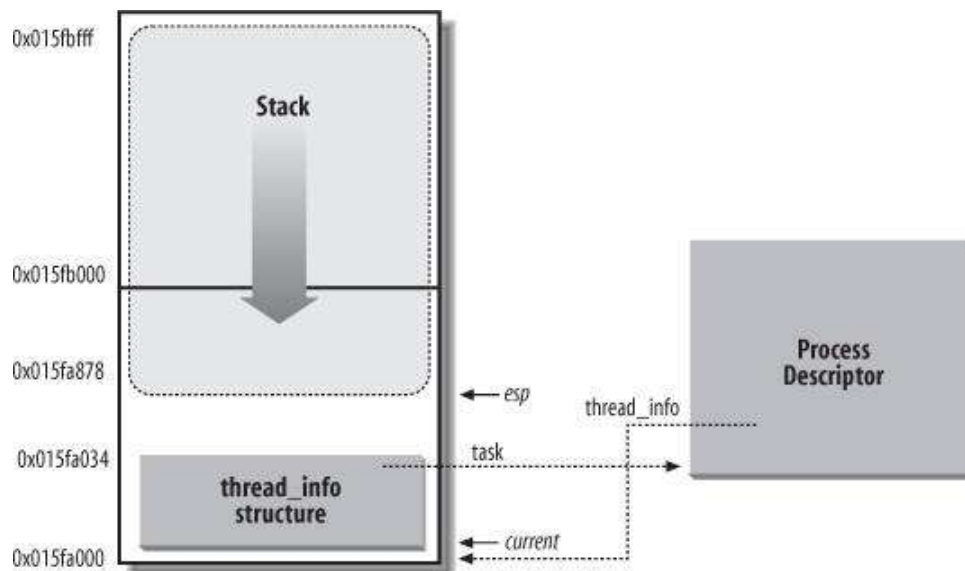
Processes are dynamic entities whose lifetimes range from a few milliseconds to months. Thus, the kernel must be able to handle many processes at the same time, and process descriptors are stored in dynamic memory rather than in the memory area permanently assigned to the kernel. For each process, Linux packs two different data structures in a single per-process memory area: a small data structure linked to the process descriptor, namely the `thread_info` structure, and the Kernel Mode process stack. The length of this memory area is usually 8,192 bytes (two page frames). For reasons of efficiency the kernel stores the 8-KB memory area in two consecutive page frames with the first page frame aligned to a multiple of 2^{13} ; this may turn out to be a problem when little dynamic

memory is available, because the free memory may become highly fragmented (see the section "[The Buddy System Algorithm](#)" in [Chapter 8](#)). Therefore, in the 80x86 architecture the kernel can be configured at compilation time so that the memory area including stack and `thread_info` structure spans a single page frame (4,096 bytes).

In the section "[Segmentation in Linux](#)" in [Chapter 2](#), we learned that a process in Kernel Mode accesses a stack contained in the kernel data segment, which is different from the stack used by the process in User Mode. Because kernel control paths make little use of the stack, only a few thousand bytes of kernel stack are required. Therefore, 8 KB is ample space for the stack and the `thread_info` structure. However, when stack and `thread_info` structure are contained in a single page frame, the kernel uses a few additional stacks to avoid the overflows caused by deeply nested interrupts and exceptions (see [Chapter 4](#)).

[Figure 3-2](#) shows how the two data structures are stored in the 2-page (8 KB) memory area. The `thread_info` structure resides at the beginning of the memory area, and the stack grows downward from the end. The figure also shows that the `thread_info` structure and the `task_struct` structure are mutually linked by means of the fields `task` and `thread_info`, respectively.

Figure 3-2. Storing the `thread_info` structure and the process kernel stack in two page frames



The `esp` register is the CPU stack pointer, which is used to address the stack's top location. On 80x86 systems, the stack starts at the end and grows toward the beginning of the memory area. Right after switching from User Mode to Kernel Mode, the kernel stack of a process is always empty, and therefore the `esp` register points to the byte immediately following the stack.

The value of the `esp` is decreased as soon as data is written into the stack. Because the `thread_info` structure is 52 bytes long, the kernel stack can expand up to 8,140 bytes.

The C language allows the `thread_info` structure and the kernel stack of a process to be conveniently represented by means of the following union construct:

```
union thread_union {
    struct thread_info thread_info;
    unsigned long stack[2048]; /* 1024 for 4KB stacks */
};
```

The `thread_info` structure shown in [Figure 3-2](#) is stored starting at address `0x015fa000`, and the stack is stored starting at address `0x015fc000`. The value of the `esp` register points to the current top of the stack at `0x015fa878`.

The kernel uses the `alloc_thread_info` and `free_thread_info` macros to allocate and release the memory area storing a `thread_info` structure and a kernel stack.

3.2.2.2. Identifying the current process

The close association between the `thread_info` structure and the Kernel Mode stack just described offers a key benefit in terms of efficiency: the kernel can easily obtain the address of the `thread_info` structure of the process currently running on a CPU from the value of the `esp` register. In fact, if the `thread_union` structure is 8 KB (2^{13} bytes) long, the kernel masks out the 13 least significant bits of `esp` to obtain the base address of the `thread_info` structure; on the other hand, if the `thread_union` structure is 4 KB long, the kernel masks out the 12 least significant bits of `esp`. This is done by the `current_thread_info( )` function, which produces assembly language instructions like the following:

```
movl $0xfffffe000,%ecx /* or 0xfffff000 for 4KB stacks */
andl %esp,%ecx
movl %ecx,p
```

After executing these three instructions, `p` contains the `thread_info` structure pointer of the process running on the CPU that executes the instruction.

Most often the kernel needs the address of the process descriptor rather than the address of the `thread_info` structure. To get the process descriptor pointer of the process currently running on a CPU, the kernel makes use of the `current` macro, which is essentially equivalent to `current_thread_info( )->task` and produces assembly language instructions like the following:

```
movl $0xfffffe000,%ecx /* or 0xfffff000 for 4KB stacks */
andl %esp,%ecx
movl (%ecx),p
```

Because the `task` field is at offset 0 in the `thread_info` structure, after executing these three instructions `p` contains the process descriptor pointer of the process running on the CPU.

The `current` macro often appears in kernel code as a prefix to fields of the process

descriptor. For example, `current->pid` returns the process ID of the process currently running on the CPU.

Another advantage of storing the process descriptor with the stack emerges on multiprocessor systems: the correct current process for each hardware processor can be derived just by checking the stack, as shown previously. Earlier versions of Linux did not store the kernel stack and the process descriptor together. Instead, they were forced to introduce a global static variable called `current` to identify the process descriptor of the running process. On multiprocessor systems, it was necessary to define `current` as an arrayone element for each available CPU.

3.2.2.3. Doubly linked lists

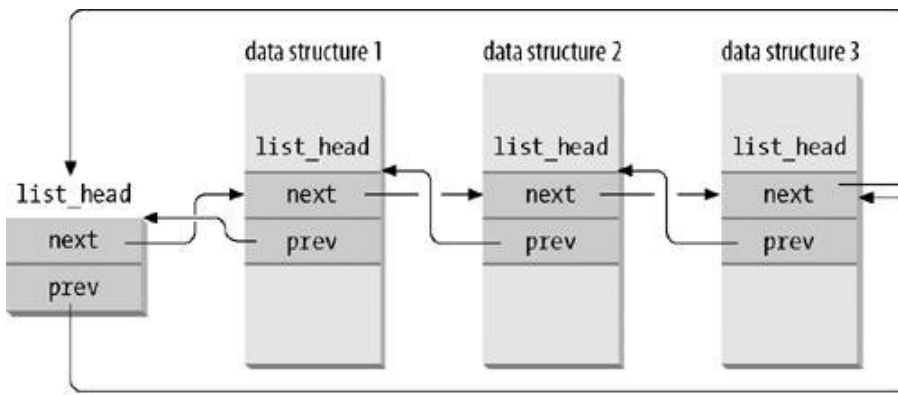
Before moving on and describing how the kernel keeps track of the various processes in the system, we would like to emphasize the role of special data structures that implement doubly linked lists.

For each list, a set of primitive operations must be implemented: initializing the list, inserting and deleting an element, scanning the list, and so on. It would be both a waste of programmers' efforts and a waste of memory to replicate the primitive operations for each different list.

Therefore, the Linux kernel defines the `list_head` data structure, whose only fields `next` and `prev` represent the forward and back pointers of a generic doubly linked list element, respectively. It is important to note, however, that the pointers in a `list_head` field store the addresses of other `list_head` fields rather than the addresses of the whole data structures in which the `list_head` structure is included; see [Figure 3-3 \(a\)](#).

A new list is created by using the `LIST_HEAD(list_name)` macro. It declares a new variable named `list_name` of type `list_head`, which is a dummy first element that acts as a placeholder for the head of the new list, and initializes the `prev` and `next` fields of the `list_head` data structure so as to point to the `list_name` variable itself; see [Figure 3-3 \(b\)](#).

Figure 3-3. Doubly linked lists built with `list_head` data structures



(a) a doubly linked list with three elements



(b) an empty doubly linked list

Several functions and macros implement the primitives, including those shown in Table [Table 3-1](#).

Table 3-1. List handling functions and macros

Name	Description
<code>list_add(n,p)</code>	Inserts an element pointed to by <code>n</code> right after the specified element pointed to by <code>p</code> . (To insert <code>n</code> at the beginning of the list, set <code>p</code> to the address of the list head.)
<code>list_add_tail(n,p)</code>	Inserts an element pointed to by <code>n</code> right before the specified element pointed to by <code>p</code> . (To insert <code>n</code> at the end of the list, set <code>p</code> to the address of the list head.)
<code>list_del(p)</code>	Deletes an element pointed to by <code>p</code> . (There is no need to specify the head of the list.)
<code>list_empty(p)</code>	


```
list_entry(p,t,m)
```

```
list_for_each(p,h)
```

```
list_for_each_entry(p,h,m)
```

Checks if the list specified by the address `p` of its head is empty. Returns the address of the data structure of type `t` in which the `list_head` field that has the name `m` and the address `p` is included.

Scans the elements of the list specified by the address `h` of the head; in each iteration, a pointer to the `list_head` structure of the list element is returned in `p`.

Similar to `list_for_each`, but returns the address of the data structure embedding the `list_head` structure rather than the address of the `list_head` structure itself.

The Linux kernel 2.6 sports another kind of doubly linked list, which mainly differs from a `list_head` list because it is not circular; it is mainly used for hash tables, where space is important, and finding the the last element in constant time is not. The list head is stored in an `hlist_head` data structure, which is simply a pointer to the first element in the list (NULL if the list is empty). Each element is represented by an `hlist_node` data structure, which includes a pointer `next` to the next element, and a pointer `pprev` to the `next` field of the previous element. Because the list is not circular, the `pprev` field of the first element and the `next` field of the last element are set to NULL. The list can be handled by means of several helper functions and macros similar to those listed in [Table 3-1](#): `hlist_add_head( )`, `hlist_del( )`, `hlist_empty( )`, `hlist_entry`, `hlist_for_each_entry`, and so on.

3.2.2.4. The process list

The first example of a doubly linked list we will examine is the process list, a list that links together all existing process descriptors. Each `task_struct` structure includes a `tasks` field of type `list_head` whose `prev` and `next` fields point, respectively, to the previous and to the next `task_struct` element.

The head of the process list is the `init_task` `task_struct` descriptor; it is the process descriptor of the so-called process 0 or swapper (see the section "[Kernel Threads](#)" later in this chapter). The `tasks->prev`

field of `init_task` points to the `tasks` field of the process descriptor inserted last in the list.

The `SET_LINKS` and `REMOVE_LINKS` macros are used to insert and to remove a process descriptor in the process list, respectively. These macros also take care of the parenthood relationship of the process (see the section "[How Processes Are Organized](#)" later in this chapter).

Another useful macro, called `for_each_process`, scans the whole process list. It is defined as:

```
#define for_each_process(p) \
    for (p=&init_task; (p=list_entry((p)->tasks.next, \
                                     struct task_struct, tasks) \
                                   ) != &init_task; )
```

The macro is the loop control statement after which the kernel programmer supplies the loop. Notice how the `init_task` process descriptor just plays the role of list header. The macro starts by moving past `init_task` to the next task and continues until it reaches `init_task` again (thanks to the circularity of the list). At each iteration, the variable passed as the argument of the macro contains the address of the currently scanned process descriptor, as returned by the `list_entry` macro.

3.2.2.5. The lists of `TASK_RUNNING` processes

When looking for a new process to run on a CPU, the kernel has to consider only the runnable processes (that is, the processes in the `TASK_RUNNING` state).

Earlier Linux versions put all runnable processes in the same list called `runqueue`. Because it would be too costly to maintain the list ordered according to process priorities, the earlier schedulers were compelled to scan the whole list in order to select the "best" runnable process.

Linux 2.6 implements the `runqueue` differently. The aim is to allow the scheduler to select the best runnable process in constant time, independently of the number of runnable processes. We'll defer to [Chapter 7](#) a detailed description of this new kind of `runqueue`, and we'll provide here only some basic information.

The trick used to achieve the scheduler speedup consists of splitting the `runqueue` in many lists of runnable processes, one list per process priority. Each `task_struct` descriptor includes a `run_list` field of type `list_head`. If the process priority is equal to `k` (a value ranging between 0 and 139), the `run_list` field links the process descriptor into the list of runnable processes having priority `k`. Furthermore, on a multiprocessor system, each CPU has its own `runqueue`, that is, its own set of lists of processes. This is a classic example of making a data structures more complex to improve performance: to make scheduler operations more efficient, the `runqueue` list has been split into 140 different lists!

As we'll see, the kernel must preserve a lot of data for every `runqueue` in the system; however, the main data structures of a `runqueue` are the lists of process descriptors belonging to the `runqueue`; all these lists are implemented by a single `prio_array_t` data structure, whose fields are shown in [Table 3-2](#).

Table 3-2. The fields of the `prio_array_t` data structure

Type

Field

Description

int

nr_active

The number of process descriptors linked into the lists

unsigned long [5]

bitmap

A priority bitmap: each flag is set if and only if the corresponding priority list is not empty

struct list_head [140]

queue

The 140 heads of the priority lists

The `enqueue_task(p, array)` function inserts a process descriptor into a runqueue list; its code is essentially equivalent to:

```
list_add_tail(&p->run_list, &array->queue[p->prio]);
__set_bit(p->prio, array->bitmap);
array->nr_active++;
p->array = array;
```

The `prio` field of the process descriptor stores the dynamic priority of the process, while the `array` field is a pointer to the `prio_array_t` data structure of its current runqueue. Similarly, the `dequeue_task(p, array)` function removes a process descriptor from a runqueue list.

3.2.3. Relationships Among Processes

Processes created by a program have a parent/child relationship. When a process creates multiple children , these children have sibling relationships. Several fields must be introduced in a process descriptor to represent these relationships; they are listed in [Table 3-3](#) with respect to a given process P. Processes 0 and 1 are created by the kernel; as we'll see later in the chapter, process 1 (init) is the ancestor of all other processes.

Table 3-3. Fields of a process descriptor used to express parenthood relationships

Field name

Description

real_parent

Points to the process descriptor of the process that created P or to the descriptor of process 1 (init) if the parent process no longer exists. (Therefore, when a user starts a background process and exits the shell, the

background process becomes the child of init.)

parent

Points to the current parent of P (this is the process that must be signaled when the child process terminates); its value usually coincides with that of `real_parent`. It may occasionally differ, such as when another process issues a `ptrace( )` system call requesting that it be allowed to monitor P (see the section "Execution Tracing" in Chapter 20).

children

The head of the list containing all children created by P.

sibling

The pointers to the next and previous elements in the list of the sibling processes, those that have the same parent as P.

Figure 3-4 illustrates the parent and sibling relationships of a group of processes. Process P0 successively created P1, P2, and P3. Process P3, in turn, created process P4.

Furthermore, there exist other relationships among processes: a process can be a leader of a process group or of a login session (see "Process Management" in Chapter 1), it can be a leader of a thread group (see "Identifying a Process" earlier in this chapter), and it can also trace the execution of other processes (see the section "Execution Tracing" in Chapter 20). Table 3-4 lists the fields of the process descriptor that establish these relationships between a process P and the other processes.

Table 3-4. The fields of the process descriptor that establish non-parenthood relationships

Field name

Description

`group_leader`

Process descriptor pointer of the group leader of P

`signal->pgrp`

PID of the group leader of P

`tgid`

PID of the thread group leader of P

`signal->session`

PID of the login session leader of P

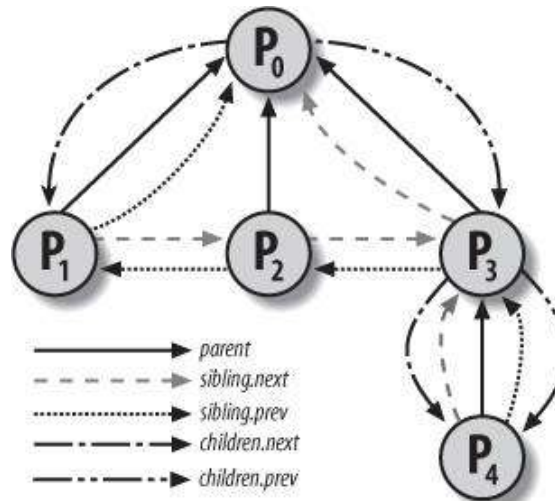
`ptrace_children`

The head of a list containing all children of P being traced by a debugger

`ptrace_list`

The pointers to the next and previous elements in the real parent's list of traced processes (used when P is being traced)

Figure 3-4. Parenthood relationships among five processes



3.2.3.1. The pidhash table and chained lists

In several circumstances, the kernel must be able to derive the process descriptor pointer corresponding to a PID. This occurs, for instance, in servicing the `kill( )` system call. When process P1 wishes to send a signal to another process, P2, it invokes the `kill( )` system call specifying the PID of P2 as the parameter. The kernel derives the process descriptor pointer from the PID and then extracts the pointer to the data structure that records the pending signals from P2's process descriptor.

Scanning the process list sequentially and checking the `pid` fields of the process descriptors is feasible but rather inefficient. To speed up the search, four hash tables have been introduced. Why multiple hash tables? Simply because the process descriptor includes fields that represent different types of PID (see [Table 3-5](#)), and each type of PID requires its own hash table.

Table 3-5. The four hash tables and corresponding fields in the process descriptor

Hash table type

Field name

Description

`PIDTYPE_PID`

pid

PID of the process

PIDTYPE_TGID

tgid

PID of thread group leader process

PIDTYPE_PGID

pgrp

PID of the group leader process

PIDTYPE_SID

session

PID of the session leader process

The four hash tables are dynamically allocated during the kernel initialization phase, and their addresses are stored in the `pid_hash` array. The size of a single hash table depends on the amount of available RAM; for example, for systems having 512 MB of RAM, each hash table is stored in four page frames and includes 2,048 entries.

The PID is transformed into a table index using the `pid_hashfn` macro, which expands to:

```
#define pid_hashfn(x) hash_long((unsigned long) x, pidhash_shift)
```

The `pidhash_shift` variable stores the length in bits of a table index (11, in our example). The `hash_long( )` function is used by many hash functions; on a 32-bit architecture it is essentially equivalent to:

```
unsigned long hash_long(unsigned long val, unsigned int bits)
{
    unsigned long hash = val * 0x9e370001UL;
    return hash >> (32 - bits);
}
```

Because in our example `pidhash_shift` is equal to 11, `pid_hashfn` yields values ranging between 0 and $2^{11} - 1 = 2047$.



The Magic Constant

You might wonder where the 0x9e370001 constant (= 2,654,404,609) comes from. This hash function is based on a multiplication of the index by a suitable large number, so that the result overflows and the value remaining in the 32-bit variable can be considered as the result of a modulus operation. Knuth suggested that good results are obtained when the large multiplier is a prime approximately in golden ratio to 2^{32} (32 bit being the size of the 80x86's registers). Now, 2,654,404,609 is a prime near to $2^{32} \times (\sqrt{5} - 1)/2$ that can also be easily multiplied by additions and bit shifts, because it is equal to $2^{31} + 2^{29} - 2^{25} + 2^{22} - 2^{19} - 2^{16} + 1$.

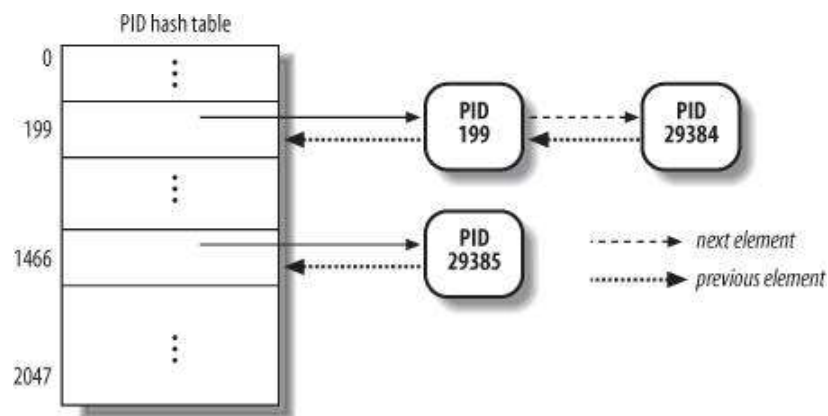
As every basic computer science course explains, a hash function does not always ensure a one-to-one correspondence between PIDs and table indexes. Two different PIDs that hash into the same table index are said to be colliding.

Linux uses chaining to handle colliding PIDs; each table entry is the head of a doubly linked list of colliding process descriptors. Figure 3-5 illustrates a PID hash table with two lists. The processes having PIDs 2,890 and 29,384 hash into the 200th element of the table, while the process having PID 29,385 hashes into the 1,466th element of the table.

Hashing with chaining is preferable to a linear transformation from PIDs to table indexes because at any given instance, the number of processes in the system is usually far below 32,768 (the maximum number of allowed PIDs). It would be a waste of storage to define a table consisting of 32,768 entries, if, at any given instance, most such entries are unused.

The data structures used in the PID hash tables are quite sophisticated, because they must keep track of the relationships between the processes. As an example, suppose that the kernel must retrieve all processes belonging to a given thread group, that is, all processes whose `tgid` field is equal to a given number. Looking in the hash table for the given thread group number returns just one process descriptor, that is, the descriptor of the thread group leader. To quickly retrieve the other processes in the group, the kernel must maintain a list of processes for each thread group. The same situation arises when looking for the processes belonging to a given login session or belonging to a given process group.

Figure 3-5. A simple PID hash table and chained lists



The PID hash tables' data structures solve all these problems, because they allow the definition of a list of processes for any PID number included in a hash table. The core data structure is an array of four `pid`

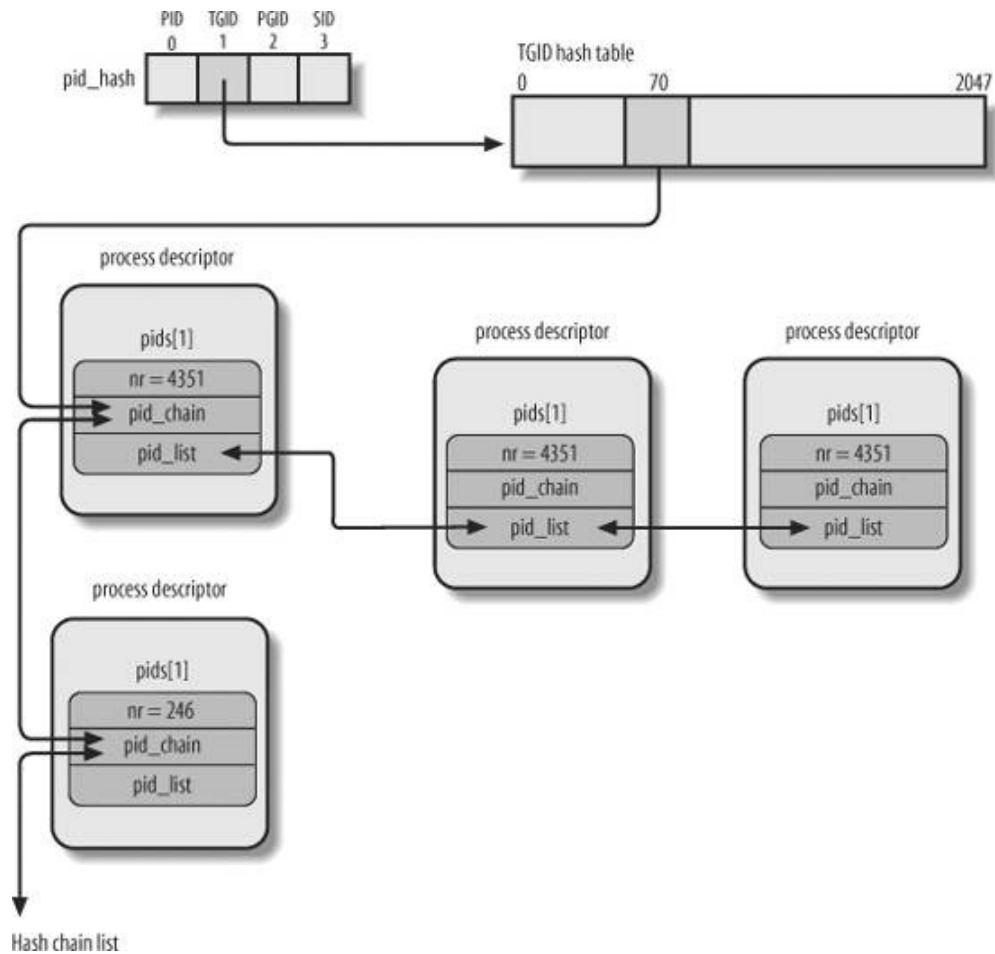
structures embedded in the `pids` field of the process descriptor; the fields of the `pid` structure are shown in [Table 3-6](#).

Table 3-6. The fields of the `pid` data structures

Type
Name
Description
<code>int</code>
<code>nr</code>
The PID number
<code>struct hlist_node</code>
<code>pid_chain</code>
The links to the next and previous elements in the hash chain list
<code>struct list_head</code>
<code>pid_list</code>
The head of the per-PID list

[Figure 3-6](#) shows an example based on the `PIDTYPE_TGID` hash table. The second entry of the `pid_hash` array stores the address of the hash table, that is, the array of `hlist_head` structures representing the heads of the chain lists. In the chain list rooted at the 71st entry of the hash table, there are two process descriptors corresponding to the PID numbers 246 and 4,351 (double-arrow lines represent a couple of forward and backward pointers). The PID numbers are stored in the `nr` field of the `pid` structure embedded in the process descriptor (by the way, because the thread group number coincides with the PID of its leader, these numbers also are stored in the `pid` field of the process descriptors). Let us consider the per-PID list of the thread group 4,351: the head of the list is stored in the `pid_list` field of the process descriptor included in the hash table, while the links to the next and previous elements of the per-PID list also are stored in the `pid_list` field of each list element.

Figure 3-6. The PID hash tables



The following functions and macros are used to handle the PID hash tables:

`do_each_task_pid(nr, type, task)`

`while_each_task_pid(nr, type, task)`

Mark begin and end of a do-while loop that iterates over the per-PID list associated with the PID number `nr` of type `type`; in any iteration, `task` points to the process descriptor of the currently scanned element.

`find_task_by_pid_type(type, nr)`

Looks for the process having PID `nr` in the hash table of type `type`. The function returns a process descriptor pointer if a match is found, otherwise it returns `NULL`.

`find_task_by_pid(nr)`

Same as `find_task_by_pid_type(PIDTYPE_PID, nr)`.

`attach_pid(task, type, nr)`

Inserts the process descriptor pointed to by `task` in the PID hash table of type `type` according to the PID number `nr`; if a process descriptor having PID `nr` is already in the hash table, the function simply inserts `task` in the per-PID list of the already present process.

`detach_pid(task, type)`

Removes the process descriptor pointed to by `task` from the per-PID list of type `type` to which the descriptor belongs. If the per-PID list does not become empty, the function terminates. Otherwise, the function removes the process descriptor from the hash table of type `type`; finally, if the PID number does not occur in any other hash table, the function clears the corresponding bit in the PID bitmap, so that the number can be recycled.

`next_thread(task)`

Returns the process descriptor address of the lightweight process that follows `task` in the hash table list of type `PIDTYPE_TGID`. Because the hash table list is circular, when applied to a conventional process the macro returns the descriptor address of the process itself.

3.2.4. How Processes Are Organized

The runqueue lists group all processes in a `TASK_RUNNING` state. When it comes to grouping processes in other states, the various states call for different types of treatment, with Linux opting for one of the choices shown in the following list.

- Processes in a `TASK_STOPPED`, `EXIT_ZOMBIE`, or `EXIT_DEAD` state are not linked in specific lists. There is no need to group processes in any of these three states, because stopped, zombie, and dead processes are accessed only via PID or via linked lists of the child processes for a particular parent.
- Processes in a `TASK_INTERRUPTIBLE` or `TASK_UNINTERRUPTIBLE` state are subdivided into many classes, each of which corresponds to a specific event. In this case, the process state does not provide enough information to retrieve the process quickly, so it is necessary to introduce additional lists of processes. These are called wait queues and are discussed next.

3.2.4.1. Wait queues

Wait queues have several uses in the kernel, particularly for interrupt handling, process synchronization, and timing. Because these topics are discussed in later chapters, we'll just say here that a process must often wait for some event to occur, such as for a disk operation to terminate, a system resource to be released, or a fixed interval of time to elapse. Wait queues implement conditional waits on events: a process wishing to wait for a specific event places itself in the proper wait queue and relinquishes control. Therefore, a wait queue represents a set of sleeping processes, which are woken up by the kernel when some condition becomes true.

Wait queues are implemented as doubly linked lists whose elements include pointers to process descriptors. Each wait queue is identified by a wait queue head, a data structure of type `wait_queue_head_t`:

```
struct __wait_queue_head {
    spinlock_t lock;
    struct list_head task_list;
};
typedef struct __wait_queue_head wait_queue_head_t;
```

Because wait queues are modified by interrupt handlers as well as by major kernel functions, the doubly linked lists must be protected from concurrent accesses, which could induce unpredictable results (see [Chapter 5](#)). Synchronization is achieved by the `lock` spin lock in the wait queue head. The `task_list` field is the head of the list of waiting processes.

Elements of a wait queue list are of type `wait_queue_t`:

```
struct __wait_queue {
    unsigned int flags;
    struct task_struct * task;
    wait_queue_func_t func;
    struct list_head task_list;
};
typedef struct __wait_queue wait_queue_t;
```

Each element in the wait queue list represents a sleeping process, which is waiting for some event to occur; its descriptor address is stored in the `task` field. The `task_list` field contains the pointers that link this element to the list of processes waiting for the same event.

However, it is not always convenient to wake up all sleeping processes in a wait queue. For instance, if two or more processes are waiting for exclusive access to some resource to be released, it makes sense to wake up just one process in the wait queue. This process takes the resource, while the other processes continue to sleep. (This avoids a problem known as the "thundering herd," with which multiple processes are wakened only to race for a resource that can be accessed by one of them, with the result that remaining processes must once more be put back to sleep.)

Thus, there are two kinds of sleeping processes: exclusive processes (denoted by the value 1 in the `flags` field of the corresponding wait queue element) are selectively woken up by the kernel, while nonexclusive processes (denoted by the value 0 in the `flags` field) are always woken up by the kernel when the event occurs. A process waiting for a resource that can be granted to just one process at a time is a typical exclusive process. Processes waiting for an event that may concern any of them are nonexclusive. Consider, for instance, a group of processes that are waiting for the termination of a group of disk block transfers: as soon as the transfers complete, all waiting processes must be woken up. As we'll see next, the `func` field of a wait queue element is used to specify how the processes sleeping in the wait queue should be woken up.

3.2.4.2. Handling wait queues

A new wait queue head may be defined by using the `DECLARE_WAIT_QUEUE_HEAD(name)` macro, which statically declares a new wait queue head variable called `name` and initializes its `lock` and `task_list` fields. The `init_waitqueue_head( )` function may be used to initialize a wait queue head variable that was allocated dynamically.

The `init_waitqueue_entry(q, p)` function initializes a `wait_queue_t` structure `q` as follows:

```
q->flags = 0;
q->task = p;
q->func = default_wake_function;
```

The nonexclusive process `p` will be awakened by `default_wake_function( )`, which is a simple

wrapper for the `TRY_to_wake_up( )` function discussed in [Chapter 7](#).

Alternatively, the `DEFINE_WAIT` macro declares a new `wait_queue_t` variable and initializes it with the descriptor of the process currently executing on the CPU and the address of the `autoremove_wake_function( )` wake-up function. This function invokes `default_wake_function( )` to awaken the sleeping process, and then removes the wait queue element from the wait queue list. Finally, a kernel developer can define a custom awakening function by initializing the wait queue element with the `init_waitqueue_func_entry( )` function.

Once an element is defined, it must be inserted into a wait queue. The `add_wait_queue( )` function inserts a nonexclusive process in the first position of a wait queue list. The `add_wait_queue_exclusive( )` function inserts an exclusive process in the last position of a wait queue list. The `remove_wait_queue( )` function removes a process from a wait queue list. The `waitqueue_active( )` function checks whether a given wait queue list is empty.

A process wishing to wait for a specific condition can invoke any of the functions shown in the following list.

- The `sleep_on( )` function operates on the current process:

```
void sleep_on(wait_queue_head_t *wq)
{
    wait_queue_t wait;
    init_waitqueue_entry(&wait, current);
    current->state = TASK_UNINTERRUPTIBLE;
    add_wait_queue(wq, &wait); /* wq points to the wait queue head */
    schedule( );
    remove_wait_queue(wq, &wait);
}
```

The function sets the state of the current process to `TASK_UNINTERRUPTIBLE` and inserts it into the specified wait queue. Then it invokes the scheduler, which resumes the execution of another process. When the sleeping process is awakened, the scheduler resumes execution of the `sleep_on( )` function, which removes the process from the wait queue.

- The `interruptible_sleep_on( )` function is identical to `sleep_on( )`, except that it sets the state of the current process to `TASK_INTERRUPTIBLE` instead of setting it to `TASK_UNINTERRUPTIBLE`, so that the process also can be woken up by receiving a signal.
- The `sleep_on_timeout( )` and `interruptible_sleep_on_timeout( )` functions are similar to the previous ones, but they also allow the caller to define a time interval after which the process will be woken up by the kernel. To do this, they invoke the `schedule_timeout( )` function instead of `schedule( )` (see the section "[An Application of Dynamic Timers: the nanosleep\(\) System Call](#)" in [Chapter 6](#)).
- The `prepare_to_wait( )`, `prepare_to_wait_exclusive( )`, and `finish_wait( )` functions, introduced in Linux 2.6, offer yet another way to put the current process to sleep in a wait queue. Typically, they are used as follows:

```
DEFINE_WAIT(wait);
prepare_to_wait_exclusive(&wq, &wait, TASK_INTERRUPTIBLE);
/* wq is the head of the wait queue */
...
if (!condition)
    schedule();
finish_wait(&wq, &wait);
```

The `prepare_to_wait( )` and `prepare_to_wait_exclusive( )` functions set the process state to the value passed as the third parameter, then set the exclusive flag in the wait queue element respectively to 0 (nonexclusive) or 1 (exclusive), and finally insert the wait queue element `wq` into the list of the wait queue head `wq`.

As soon as the process is awakened, it executes the `finish_wait( )` function, which sets again the process state to `TASK_RUNNING` (just in case the awaking condition becomes true before invoking `schedule( )`), and removes the wait queue element from the wait queue list (unless this has already been done by the wake-up function).

- The `wait_event` and `wait_event_interruptible` macros put the calling process to sleep on a wait queue until a given condition is verified. For instance, the `wait_event(wq, condition)` macro essentially yields the following fragment:

```

DEFINE_WAIT(_ _wait);
for (;;) {
    prepare_to_wait(&wq, &_ _wait, TASK_UNINTERRUPTIBLE);
    if (condition)
        break;
    schedule( );
}
finish_wait(&wq, &_ _wait);

```

A few comments on the functions mentioned in the above list: the `sleep_on( )`-like functions cannot be used in the common situation where one has to test a condition and atomically put the process to sleep when the condition is not verified; therefore, because they are a well-known source of race conditions, their use is discouraged. Moreover, in order to insert an exclusive process into a wait queue, the kernel must make use of the `prepare_to_wait_exclusive( )` function (or just invoke `add_wait_queue_exclusive( )` directly); any other helper function inserts the process as nonexclusive. Finally, unless `DEFINE_WAIT` or `finish_wait( )` are used, the kernel must remove the wait queue element from the list after the waiting process has been awakened.

The kernel awakens processes in the wait queues, putting them in the `TASK_RUNNING` state, by means of one of the following macros: `wake_up`, `wake_up_nr`, `wake_up_all`, `wake_up_interruptible`, `wake_up_interruptible_nr`, `wake_up_interruptible_all`, `wake_up_interruptible_sync`, and `wake_up_locked`. One can understand what each of these nine macros does from its name:

- All macros take into consideration sleeping processes in the `TASK_INTERRUPTIBLE` state; if the macro name does not include the string "interruptible," sleeping processes in the `TASK_UNINTERRUPTIBLE` state also are considered.
- All macros wake all nonexclusive processes having the required state (see the previous bullet item).
- The macros whose name include the string "nr" wake a given number of exclusive processes having the required state; this number is a parameter of the macro. The macros whose names include the string "all" wake all exclusive processes having the required state. Finally, the macros whose names don't include "nr" or "all" wake exactly one exclusive process that has the required state.
- The macros whose names don't include the string "sync" check whether the priority of any of the woken processes is higher than that of the processes currently running in the systems and invoke `schedule( )` if necessary. These checks are not made by the macro whose name includes the string "sync"; as a result, execution of a high priority process might be slightly delayed.
- The `wake_up_locked` macro is similar to `wake_up`, except that it is called when the spin lock in `wait_queue_head_t` is already held.

For instance, the `wake_up` macro is essentially equivalent to the following code fragment:

```

void wake_up(wait_queue_head_t *q)
{
    struct list_head *tmp;
    wait_queue_t *curr;

    list_for_each(tmp, &q->task_list) {
        curr = list_entry(tmp, wait_queue_t, task_list);
        if (curr->func(curr, TASK_INTERRUPTIBLE|TASK_UNINTERRUPTIBLE,
                        0, NULL) && curr->flags)
            break;
    }
}

```

The `list_for_each` macro scans all items in the `q->task_list` doubly linked list, that is, all processes in the wait queue. For each item, the `list_entry` macro computes the address of the corresponding `wait_queue_t` variable. The `func` field of this variable stores the address of the wake-up function, which tries to wake up the process identified by the `task` field of the wait queue element. If a process has been effectively awakened (the function returned 1) and if the process is exclusive (`curr->flags` equal to 1), the loop terminates. Because all nonexclusive processes are always at the beginning of the doubly linked list and all exclusive processes are at the end, the function always wakes the nonexclusive processes and then wakes one exclusive process, if any exists.^[*]

[*] By the way, it is rather uncommon that a wait queue includes both exclusive and nonexclusive processes.

3.2.5. Process Resource Limits

Each process has an associated set of resource limits, which specify the amount of system resources it can use. These limits keep a user from overwhelming the system (its CPU, disk space, and so on). Linux recognizes the following resource limits illustrated in [Table 3-7](#).

The resource limits for the current process are stored in the `current->signal->rlim` field, that is, in a field of the process's signal descriptor (see the section "[Data Structures Associated with Signals](#)" in [Chapter 11](#)). The field is an array of elements of type `struct rlimit`, one for each resource limit:

```

struct rlimit {
    unsigned long rlim_cur;
    unsigned long rlim_max;
};

```

Table 3-7. Resource limits

Field name

Description

RLIMIT_AS

The maximum size of process address space, in bytes. The kernel checks this value when the process uses `malloc( )` or a related function to enlarge its address space (see the section "[The Process's Address Space](#)"

in [Chapter 9](#)).

`RLIMIT_CORE`

The maximum core dump file size, in bytes. The kernel checks this value when a process is aborted, before creating a core file in the current directory of the process (see the section "[Actions Performed upon Delivering a Signal](#)" in [Chapter 11](#)). If the limit is 0, the kernel won't create the file.

`RLIMIT_CPU`

The maximum CPU time for the process, in seconds. If the process exceeds the limit, the kernel sends it a `SIGXCPU` signal, and then, if the process doesn't terminate, a `SIGKILL` signal (see [Chapter 11](#)).

`RLIMIT_DATA`

The maximum heap size, in bytes. The kernel checks this value before expanding the heap of the process (see the section "[Managing the Heap](#)" in [Chapter 9](#)).

`RLIMIT_FSIZE`

The maximum file size allowed, in bytes. If the process tries to enlarge a file to a size greater than this value, the kernel sends it a `SIGXFSZ` signal.

`RLIMIT_LOCKS`

Maximum number of file locks (currently, not enforced).

`RLIMIT_MEMLOCK`

The maximum size of nonswappable memory, in bytes. The kernel checks this value when the process tries to lock a page frame in memory using the `mlock( )` or `mlockall( )` system calls (see the section "[Allocating a Linear Address Interval](#)" in [Chapter 9](#)).

`RLIMIT_MSGQUEUE`

Maximum number of bytes in POSIX message queues (see the section "[POSIX Message Queues](#)" in [Chapter 19](#)).

`RLIMIT_NOFILE`

The maximum number of open file descriptors. The kernel checks this value when opening a new file or duplicating a file descriptor (see [Chapter 12](#)).

`RLIMIT_NPROC`

The maximum number of processes that the user can own (see the section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)" later in this chapter).

`RLIMIT_RSS`

The maximum number of page frames owned by the process (currently, not enforced).

`RLIMIT_SIGPENDING`

The maximum number of pending signals for the process (see [Chapter 11](#)).

`RLIMIT_STACK`

The maximum stack size, in bytes. The kernel checks this value before expanding the User Mode stack of the process (see the section "[Page Fault Exception Handler](#)" in [Chapter 9](#)).

The `rlim_cur` field is the current resource limit for the resource. For example, `current->signal->rlim[RLIMIT_CPU].rlim_cur` represents the current limit on the CPU time of the running process.

The `rlim_max` field is the maximum allowed value for the resource limit. By using the `getrlimit( )` and `setrlimit( )` system calls, a user can always increase the `rlim_cur` limit of some resource up to `rlim_max`. However, only the superuser (or, more precisely, a user who has the `CAP_SYS_RESOURCE` capability) can increase the `rlim_max` field or set the `rlim_cur` field to a value greater than the corresponding `rlim_max` field.

Most resource limits contain the value `RLIM_INFINITY (0xffffffff)`, which means that no user limit is imposed on the corresponding resource (of course, real limits exist due to kernel design restrictions, available RAM, available space on disk, etc.). However, the system administrator may choose to impose stronger limits on some resources. Whenever a user logs into the system, the kernel creates a process owned by the superuser, which can invoke `setrlimit( )` to decrease the `rlim_max` and `rlim_cur` fields for a resource. The same process later executes a login shell and becomes owned by the user. Each new process created by the user inherits the content of the `rlim` array from its parent, and therefore the user cannot override the limits enforced by the administrator.

3.3. Process Switch

To control the execution of processes, the kernel must be able to suspend the execution of the process running on the CPU and resume the execution of some other process previously suspended. This activity goes variously by the names process switch, task switch, or context switch. The next sections describe the elements of process switching in Linux.

3.3.1. Hardware Context

While each process can have its own address space, all processes have to share the CPU registers. So before resuming the execution of a process, the kernel must ensure that each such register is loaded with the value it had when the process was suspended.

The set of data that must be loaded into the registers before the process resumes its execution on the CPU is called the hardware context. The hardware context is a subset of the process execution context, which includes all information needed for the process execution. In Linux, a part of the hardware context of a process is stored in the process descriptor, while the remaining part is saved in the Kernel Mode stack.

In the description that follows, we will assume the `prev` local variable refers to the process descriptor of the process being switched out and `next` refers to the one being switched in to replace it. We can thus define a process switch as the activity consisting of saving the hardware context of `prev` and replacing it with the hardware context of `next`. Because process switches occur quite often, it is important to minimize the time spent in saving and loading hardware contexts.

Old versions of Linux took advantage of the hardware support offered by the 80x86 architecture and performed a process switch through a `far jmp` instruction^[*] to the selector of the Task State Segment Descriptor of the `next` process. While executing the instruction, the CPU performs a hardware context switch by automatically saving the old hardware context and loading a new one. But Linux 2.6 uses software to perform a process switch for the following reasons:

[*] `far jmp` instructions modify both the `cs` and `eip` registers, while simple `jmp` instructions modify only `eip`.

- Step-by-step switching performed through a sequence of `mov` instructions allows better control over the validity of the data being loaded. In particular, it is possible to check the values of the `ds` and `es` segmentation registers, which might have been forged by a malicious user. This type of checking is not possible when using a single `far jmp` instruction.
- The amount of time required by the old approach and the new approach is about the same. However, it is not possible to optimize a hardware context switch, while there might be room for improving the current switching code.

Process switching occurs only in Kernel Mode. The contents of all registers used by a process in User Mode have already been saved on the Kernel Mode stack before performing process switching (see [Chapter 4](#)). This includes the contents of the `ss` and `esp` pair that specifies the User Mode stack pointer address.

3.3.2. Task State Segment

The 80x86 architecture includes a specific segment type called the Task State Segment (TSS), to store hardware contexts. Although Linux doesn't use hardware context switches, it is nonetheless forced to set up a TSS for each distinct CPU in the system. This is done for two main reasons:

- When an 80x86 CPU switches from User Mode to Kernel Mode, it fetches the address of the Kernel Mode stack from the TSS (see the sections "[Hardware Handling of Interrupts and Exceptions](#)" in [Chapter 4](#) and "[Issuing a System Call via the sysenter Instruction](#)" in [Chapter 10](#)).
- When a User Mode process attempts to access an I/O port by means of an `in` or `out` instruction, the CPU may need to access an I/O Permission Bitmap stored in the TSS to verify whether the process is allowed to address the port.

More precisely, when a process executes an `in` or `out` I/O instruction in User Mode, the control unit performs the following operations:

1. It checks the 2-bit IOPL field in the `eflags` register. If it is set to 3, the control unit executes the I/O instructions. Otherwise, it performs the next check.
2. It accesses the `tr` register to determine the current TSS, and thus the proper I/O Permission Bitmap.
3. It checks the bit of the I/O Permission Bitmap corresponding to the I/O port specified in the I/O instruction. If it is cleared, the instruction is executed; otherwise, the control unit raises a "General protection " exception.

The `tss_struct` structure describes the format of the TSS. As already mentioned in [Chapter 2](#), the `init_tss` array stores one TSS for each CPU on the system. At each process switch, the kernel updates some fields of the TSS so that the corresponding CPU's control unit may safely retrieve the information it needs. Thus, the TSS reflects the privilege of the current process on the CPU, but there is no need to maintain TSSs for processes when they're not running.

Each TSS has its own 8-byte Task State Segment Descriptor (TSSD). This descriptor includes a 32-bit `Base` field that points to the TSS starting address and a 20-bit `Limit` field. The `S` flag of a TSSD is cleared to denote the fact that the corresponding TSS is a System Segment (see the section "[Segment Descriptors](#)" in [Chapter 2](#)).

The `Type` field is set to either 9 or 11 to denote that the segment is actually a TSS. In the Intel's original design, each process in the system should refer to its own TSS; the second least significant bit of the `Type` field is called the Busy bit; it is set to 1 if the process is being executed by a CPU, and to 0 otherwise. In Linux design, there is just one TSS for each CPU, so the Busy bit is always set to 1.

The TSSDs created by Linux are stored in the Global Descriptor Table (GDT), whose base address is stored in the `gdtr` register of each CPU. The `tr` register of each CPU contains the TSSD Selector of the corresponding TSS. The register also includes two hidden, nonprogrammable fields: the `Base` and `Limit` fields of the TSSD. In this way, the processor can address the TSS directly without having to retrieve the TSS address from the GDT.

3.3.2.1. The thread field

At every process switch, the hardware context of the process being replaced must be saved somewhere. It cannot be saved on the TSS, as in the original Intel design, because Linux uses a single TSS for each

processor, instead of one for every process.

Thus, each process descriptor includes a field called `thread` of type `thread_struct`, in which the kernel saves the hardware context whenever the process is being switched out. As we'll see later, this data structure includes fields for most of the CPU registers, except the general-purpose registers such as `eax`, `ebx`, etc., which are stored in the Kernel Mode stack.

3.3.3. Performing the Process Switch

A process switch may occur at just one well-defined point: the `schedule( )` function, which is discussed at length in [Chapter 7](#). Here, we are only concerned with how the kernel performs a process switch.

Essentially, every process switch consists of two steps:

1. Switching the Page Global Directory to install a new address space; we'll describe this step in [Chapter 9](#).
2. Switching the Kernel Mode stack and the hardware context, which provides all the information needed by the kernel to execute the new process, including the CPU registers.

Again, we assume that `prev` points to the descriptor of the process being replaced, and `next` to the descriptor of the process being activated. As we'll see in [Chapter 7](#), `prev` and `next` are local variables of the `schedule( )` function.

3.3.3.1. The `switch_to` macro

The second step of the process switch is performed by the `switch_to` macro. It is one of the most hardware-dependent routines of the kernel, and it takes some effort to understand what it does.

First of all, the macro has three parameters, called `prev`, `next`, and `last`. You might easily guess the role of `prev` and `next`: they are just placeholders for the local variables `prev` and `next`, that is, they are input parameters that specify the memory locations containing the descriptor address of the process being replaced and the descriptor address of the new process, respectively.

What about the third parameter, `last`? Well, in any process switch three processes are involved, not just two. Suppose the kernel decides to switch off process A and to activate process B. In the `schedule( )` function, `prev` points to A's descriptor and `next` points to B's descriptor. As soon as the `switch_to` macro deactivates A, the execution flow of A freezes.

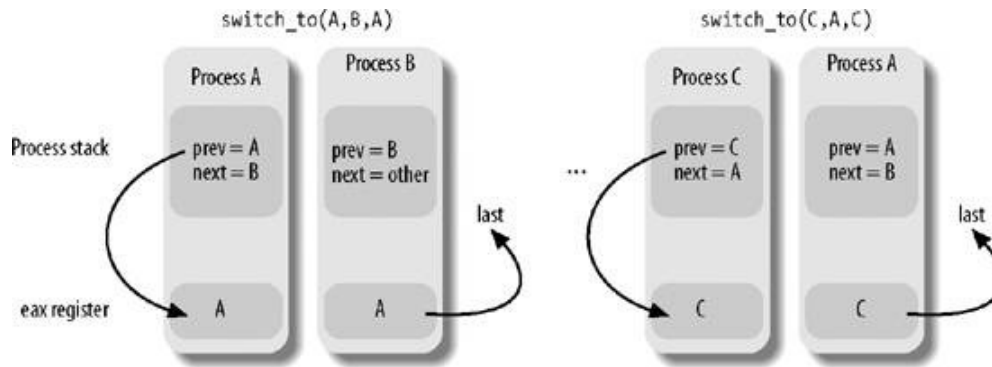
Later, when the kernel wants to reactivate A, it must switch off another process C (in general, this is different from B) by executing another `switch_to` macro with `prev` pointing to C and `next` pointing to A. When A resumes its execution flow, it finds its old Kernel Mode stack, so the `prev` local variable points to A's descriptor and `next` points to B's descriptor. The scheduler, which is now executing on behalf of process A, has lost any reference to C. This reference, however, turns out to be useful to complete the process switching (see [Chapter 7](#) for more details).

The last parameter of the `switch_to` macro is an output parameter that specifies a memory location in which the macro writes the descriptor address of process C (of course, this is done after A resumes its execution). Before the process switching, the macro saves in the `eax` CPU register the content of the variable identified by the first input parameter `prev` that is, the `prev` local variable allocated on the Kernel Mode stack of A. After the process switching, when A has resumed its execution, the macro writes the content of the `eax` CPU register in the memory location of A identified by the third output parameter `last`. Because the CPU register doesn't change across the process switch, this memory location receives the address of C's

descriptor. In the current implementation of `schedule()`, the last parameter identifies the `prev` local variable of A, so `prev` is overwritten with the address of C.

The contents of the Kernel Mode stacks of processes A, B, and C are shown in Figure 3-7, together with the values of the `eax` register; be warned that the figure shows the value of the `prev` local variable before its value is overwritten with the contents of the `eax` register.

Figure 3-7. Preserving the reference to process C across a process switch



The `switch_to` macro is coded in extended inline assembly language that makes for rather complex reading: in fact, the code refers to registers by means of a special positional notation that allows the compiler to freely choose the general-purpose registers to be used. Rather than follow the cumbersome extended inline assembly language, we'll describe what the `switch_to` macro typically does on an 80x86 microprocessor by using standard assembly language:

1. Saves the values of `prev` and `next` in the `eax` and `edx` registers, respectively:

```
movl prev, %eax
movl next, %edx
```

2. Saves the contents of the `eflags` and `ebp` registers in the `prev` Kernel Mode stack. They must be saved because the compiler assumes that they will stay unchanged until the end of `switch_to`:

```
pushfl
pushl %ebp
```

3. Saves the content of `esp` in `prev->thread.esp` so that the field points to the top of the `prev` Kernel Mode stack:

```
movl %esp, 484(%eax)
```

The `484(%eax)` operand identifies the memory cell whose address is the contents of `eax` plus 484.

4. Loads `next->thread.esp` in `esp`. From now on, the kernel operates on the Kernel Mode stack of `next`, so this instruction performs the actual process switch from `prev` to `next`. Because the address of a process descriptor is closely related to that of the Kernel Mode stack (as explained in the section "Identifying a Process" earlier in this chapter), changing the kernel stack means changing the current process:

```
movl 484(%edx), %esp
```

5. Saves the address labeled 1 (shown later in this section) in `prev->thread.eip`. When the process being replaced resumes its execution, the process executes the instruction labeled as 1:

```
movl $1f, 480(%eax)
```

6. On the Kernel Mode stack of `next`, the macro pushes the `next->thread.eip` value, which, in most cases, is the address labeled as 1:

```
pushl 480(%edx)
```

7. Jumps to the `__switch_to( )` C function (see next):

```
jmp __switch_to
```

8. Here process A that was replaced by B gets the CPU again: it executes a few instructions that restore the contents of the `eflags` and `ebp` registers. The first of these two instructions is labeled as 1:

```
1:
    popl %ebp
    popfl
```

Notice how these `pop` instructions refer to the kernel stack of the `prev` process. They will be executed when the scheduler selects `prev` as the new process to be executed on the CPU, thus invoking `switch_to` with `prev` as the second parameter. Therefore, the `esp` register points to the `prev`'s Kernel Mode stack.

9. Copies the content of the `eax` register (loaded in step 1 above) into the memory location identified by the third parameter `last` of the `switch_to` macro:

```
movl %eax, last
```

As discussed earlier, the `eax` register points to the descriptor of the process that has just been replaced.^[*]

[*] As stated earlier in this section, the current implementation of the `schedule( )` function reuses the `prev` local variable, so that the assembly language instruction looks like `movl %eax, prev`.

3.3.3.2. The `__switch_to( )` function

The `__switch_to( )` function does the bulk of the process switch started by the `switch_to( )` macro. It acts on the `prev_p` and `next_p` parameters that denote the former process and the new process. This function call is different from the average function call, though, because `__switch_to( )` takes the `prev_p` and `next_p` parameters from the `eax` and `edx` registers (where we saw they were stored), not

from the stack like most functions. To force the function to go to the registers for its parameters, the kernel uses the `__attribute__((regparm(3)))` and `regparm` keywords, which are nonstandard extensions of the C language implemented by the `gcc` compiler. The `__switch_to()` function is declared in the `include/asm-i386/system.h` header file as follows:

```
__switch_to(struct task_struct *prev_p,
            struct task_struct *next_p)
__attribute__((regparm(3)));
```

The steps performed by the function are the following:

1. Executes the code yielded by the `__unlazy_fpu()` macro (see the section "[Saving and Loading the FPU, MMX, and XMM Registers](#)" later in this chapter) to optionally save the contents of the FPU, MMX, and XMM registers of the `prev_p` process.

```
__unlazy_fpu(prev_p);
```

2. Executes the `smp_processor_id()` macro to get the index of the local CPU, namely the CPU that executes the code. The macro gets the index from the `cpu` field of the `thread_info` structure of the current process and stores it into the `cpu` local variable.
3. Loads `next_p->thread.esp0` in the `esp0` field of the TSS relative to the local CPU; as we'll see in the section "[Issuing a System Call via the sysenter Instruction](#)" in [Chapter 10](#), any future privilege level change from User Mode to Kernel Mode raised by a `sysenter` assembly instruction will copy this address in the `esp` register:

```
init_tss[cpu].esp0 = next_p->thread.esp0;
```

4. Loads in the Global Descriptor Table of the local CPU the Thread-Local Storage (TLS) segments used by the `next_p` process; the three Segment Selectors are stored in the `tls_array` array inside the process descriptor (see the section "[Segmentation in Linux](#)" in [Chapter 2](#)).

```
cpu_gdt_table[cpu][6] = next_p->thread.tls_array[0];
cpu_gdt_table[cpu][7] = next_p->thread.tls_array[1];
cpu_gdt_table[cpu][8] = next_p->thread.tls_array[2];
```

5. Stores the contents of the `fs` and `gs` segmentation registers in `prev_p->thread.fs` and `prev_p->thread.gs`, respectively; the corresponding assembly language instructions are:

```
movl %fs, 40(%esi)
movl %gs, 44(%esi)
```

The `esi` register points to the `prev_p->thread` structure.

6. If the `fs` or the `gs` segmentation register have been used either by the `prev_p` or by the `next_p` process (i.e., if they have a nonzero value), loads into these registers the values stored in the `thread_struct` descriptor of the `next_p` process. This step logically complements the actions performed in the previous step. The main assembly language instructions are:

```
movl 40(%ebx), %fs
movl 44(%ebx), %gs
```

The `ebx` register points to the `next_p->thread` structure. The code is actually more intricate, as an exception might be raised by the CPU when it detects an invalid segment register value. The code takes this possibility into account by adopting a "fix-up" approach (see the section "[Dynamic Address Checking: The Fix-up Code](#)" in [Chapter 10](#)).

7. Loads six of the `dr0,...,dr7` debug registers ^[*] with the contents of the `next_p->thread.debugreg` array. This is done only if `next_p` was using the debug registers when it was suspended (that is, field `next_p->thread.debugreg[7]` is not 0). These registers need not be saved, because the `prev_p->thread.debugreg` array is modified only when a debugger wants to monitor `prev`:

[*] The 80x86 debug registers allow a process to be monitored by the hardware. Up to four breakpoint areas may be defined. Whenever a monitored process issues a linear address included in one of the breakpoint areas, an exception occurs.

```
if (next_p->thread.debugreg[7]) {
    loaddebug(&next_p->thread, 0);
    loaddebug(&next_p->thread, 1);
    loaddebug(&next_p->thread, 2);
    loaddebug(&next_p->thread, 3);
    /* no 4 and 5 */
    loaddebug(&next_p->thread, 6);
    loaddebug(&next_p->thread, 7);
}
```

8. Updates the I/O bitmap in the TSS, if necessary. This must be done when either `next_p` or `prev_p` has its own customized I/O Permission Bitmap:

```
if (prev_p->thread.io_bitmap_ptr || next_p->thread.io_bitmap_ptr)
    handle_io_bitmap(&next_p->thread, &init_tss[cpu]);
```

Because processes seldom modify the I/O Permission Bitmap, this bitmap is handled in a "lazy" mode: the actual bitmap is copied into the TSS of the local CPU only if a process actually accesses an I/O port in the current timeslice. The customized I/O Permission Bitmap of a process is stored in a buffer pointed to by the `io_bitmap_ptr` field of the `thread_info` structure. The `handle_io_bitmap( )` function sets up the `io_bitmap` field of the TSS used by the local CPU for the `next_p` process as follows:

- ◆ If the `next_p` process does not have its own customized I/O Permission Bitmap, the `io_bitmap` field of the TSS is set to the value `0x8000`.
- ◆ If the `next_p` process has its own customized I/O Permission Bitmap, the `io_bitmap` field of the TSS is set to the value `0x9000`.

The `io_bitmap` field of the TSS should contain an offset inside the TSS where the actual bitmap is stored. The `0x8000` and `0x9000` values point outside of the TSS limit and will thus cause a "General protection" exception whenever the User Mode process attempts to access an I/O port (see the section "[Exceptions](#)" in [Chapter 4](#)). The `do_general_protection( )` exception handler will check the value stored in the `io_bitmap` field: if it is `0x8000`, the function sends a `SIGSEGV` signal to the User Mode process; otherwise, if it is `0x9000`, the function copies the process bitmap (pointed to by the `io_bitmap_ptr` field in the `thread_info` structure) in the TSS of the local CPU, sets the `io_bitmap` field to the actual bitmap offset (104), and forces a new execution of the faulty assembly language instruction.

9. Terminates. The `__switch_to( )` C function ends by means of the statement:


```
return prev_p;
```

The corresponding assembly language instructions generated by the compiler are:

```
movl %edi,%eax
ret
```

The `prev_p` parameter (now in `edi`) is copied into `eax`, because by default the return value of any C function is passed in the `eax` register. Notice that the value of `eax` is thus preserved across the invocation of `__switch_to( )`; this is quite important, because the invoking `switch_to` macro assumes that `eax` always stores the address of the process descriptor being replaced.

The `ret` assembly language instruction loads the `eip` program counter with the return address stored on top of the stack. However, the `__switch_to( )` function has been invoked simply by jumping into it. Therefore, the `ret` instruction finds on the stack the address of the instruction labeled as 1, which was pushed by the `switch_to` macro. If `next_p` was never suspended before because it is being executed for the first time, the function finds the starting address of the `ret_from_fork( )` function (see the section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)" later in this chapter).

3.3.4. Saving and Loading the FPU, MMX, and XMM Registers

Starting with the Intel 80486DX, the arithmetic floating-point unit (FPU) has been integrated into the CPU. The name mathematical coprocessor continues to be used in memory of the days when floating-point computations were executed by an expensive special-purpose chip. To maintain compatibility with older models, however, floating-point arithmetic functions are performed with ESCAPE instructions, which are instructions with a prefix byte ranging between `0xd8` and `0xdf`. These instructions act on the set of floating-point registers included in the CPU. Clearly, if a process is using ESCAPE instructions, the contents of the floating-point registers belong to its hardware context and should be saved.

In later Pentium models, Intel introduced a new set of assembly language instructions into its microprocessors. They are called MMX instructions and are supposed to speed up the execution of multimedia applications. MMX instructions act on the floating-point registers of the FPU. The obvious disadvantage of this architectural choice is that programmers cannot mix floating-point instructions and MMX instructions. The advantage is that operating system designers can ignore the new instruction set, because the same facility of the task-switching code for saving the state of the floating-point unit can also be relied upon to save the MMX state.

MMX instructions speed up multimedia applications, because they introduce a single-instruction multiple-data (SIMD) pipeline inside the processor. The Pentium III model extends that SIMD capability: it introduces the SSE extensions (Streaming SIMD Extensions), which adds facilities for handling floating-point values contained in eight 128-bit registers called the XMM registers. Such registers do not overlap with the FPU and MMX registers, so SSE and FPU/MMX instructions may be freely mixed. The Pentium 4 model introduces yet another feature: the SSE2 extensions, which is basically an extension of SSE supporting higher-precision floating-point values. SSE2 uses the same set of XMM registers as SSE.

The 80x86 microprocessors do not automatically save the FPU, MMX, and XMM registers in the TSS. However, they include some hardware support that enables kernels to save these registers only when needed.

The hardware support consists of a TS (Task-Switching) flag in the `cr0` register, which obeys the following rules:

- Every time a hardware context switch is performed, the TS flag is set.
- Every time an ESCAPE, MMX, SSE, or SSE2 instruction is executed when the TS flag is set, the control unit raises a "Device not available " exception (see [Chapter 4](#)).

The TS flag allows the kernel to save and restore the FPU, MMX, and XMM registers only when really needed. To illustrate how it works, suppose that a process A is using the mathematical coprocessor. When a context switch occurs from A to B, the kernel sets the TS flag and saves the floating-point registers into the TSS of process A. If the new process B does not use the mathematical coprocessor, the kernel won't need to restore the contents of the floating-point registers. But as soon as B tries to execute an ESCAPE or MMX instruction, the CPU raises a "Device not available" exception, and the corresponding handler loads the floating-point registers with the values saved in the TSS of process B.

Let's now describe the data structures introduced to handle selective loading of the FPU, MMX, and XMM registers. They are stored in the `thread.i387` subfield of the process descriptor, whose format is described by the `i387_union` union:

```
union i387_union {
    struct i387_fsave_struct    fsave;
    struct i387_fxsave_struct   fxsave;
    struct i387_soft_struct     soft;
};
```

As you see, the field may store just one of three different types of data structures. The `i387_soft_struct` type is used by CPU models without a mathematical coprocessor; the Linux kernel still supports these old chips by emulating the coprocessor via software. We don't discuss this legacy case further, however. The `i387_fsave_struct` type is used by CPU models with a mathematical coprocessor and, optionally, an MMX unit. Finally, the `i387_fxsave_struct` type is used by CPU models featuring SSE and SSE2 extensions.

The process descriptor includes two additional flags:

- The `TS_USED_FPU` flag, which is included in the `status` field of the `thread_info` descriptor. It specifies whether the process used the FPU, MMX, or XMM registers in the current execution run.
- The `PF_USED_MATH` flag, which is included in the `flags` field of the `task_struct` descriptor. This flag specifies whether the contents of the `thread.i387` subfield are significant. The flag is cleared (not significant) in two cases, shown in the following list.
 - ◆ When the process starts executing a new program by invoking an `execve( )` system call (see [Chapter 20](#)). Because control will never return to the former program, the data currently stored in `thread.i387` is never used again.
 - ◆ When a process that was executing a program in User Mode starts executing a signal handler procedure (see [Chapter 11](#)). Because signal handlers are asynchronous with respect to the program execution flow, the floating-point registers could be meaningless to the signal handler. However, the kernel saves the floating-point registers in `thread.i387` before starting the handler and restores them after the handler terminates. Therefore, a signal handler is allowed to use the mathematical coprocessor.

3.3.4.1. Saving the FPU registers

As stated earlier, the `__switch_to( )` function executes the `__unlazy_fpu` macro, passing the process descriptor of the `prev` process being replaced as an argument. The macro checks the value of the `TS_USED_FPU` flags of `prev`. If the flag is set, `prev` has used an FPU, MMX, SSE, or SSE2 instructions; therefore, the kernel must save the relative hardware context:

```
if (prev->thread_info->status & TS_USED_FPU)
    save_init_fpu(prev);
```

The `save_init_fpu( )` function, in turn, executes essentially the following operations:

1. Dumps the contents of the FPU registers in the process descriptor of `prev` and then reinitializes the FPU. If the CPU uses SSE/SSE2 extensions, it also dumps the contents of the XMM registers and reinitializes the SSE/SSE2 unit. A couple of powerful extended inline assembly language instructions take care of everything, either:

```
asm volatile( "fxsave
%0 ; fnclex"
: "=m" (prev->thread.i387.fxsave) );
```

if the CPU uses SSE/SSE2 extensions, or otherwise:

```
asm volatile( "fnsave
%0 ; fwait"
: "=m" (prev->thread.i387.fsave) );
```

2. Resets the `TS_USED_FPU` flag of `prev`:

```
prev->thread_info->status &= ~TS_USED_FPU;
```

3. Sets the CW flag of `cr0` by means of the `stts( )` macro, which in practice yields assembly language instructions like the following:

```
movl %cr0, %eax
orl $8,%eax
movl %eax, %cr0
```

3.3.4.2. Loading the FPU registers

The contents of the floating-point registers are not restored right after the `next` process resumes execution. However, the `TS` flag of `cr0` has been set by `__unlazy_fpu( )`. Thus, the first time the `next` process tries to execute an ESCAPE, MMX, or SSE/SSE2 instruction, the control unit raises a "Device not available" exception, and the kernel (more precisely, the exception handler involved by the exception) runs the

`math_state_restore( )` function. The next process is identified by this handler as `current`.

```
void math_state_restore( )
{
    asm volatile ("clts"); /* clear the TS flag of cr0 */
    if (!(current->flags & PF_USED_MATH))
        init_fpu(current);
    restore_fpu(current);
    current->thread.status |= TS_USEDFPU;
}
```

The function clears the CW flags of `cr0`, so that further FPU, MMX, or SSE/SSE2 instructions executed by the process won't trigger the "Device not available" exception. If the contents of the `thread.i387` subfield are not significant, i.e., if the `PF_USED_MATH` flag is equal to 0, `init_fpu( )` is invoked to reset the `thread.i387` subfield and to set the `PF_USED_MATH` flag of `current` to 1. The `restore_fpu( )` function is then invoked to load the FPU registers with the proper values stored in the `thread.i387` subfield. To do this, either the `fxrstor` or the `frstor` assembly language instructions are used, depending on whether the CPU supports SSE/SSE2 extensions. Finally, `math_state_restore( )` sets the `TS_USEDFPU` flag.

3.3.4.3. Using the FPU, MMX, and SSE/SSE2 units in Kernel Mode

Even the kernel can make use of the FPU, MMX, or SSE/SSE2 units. In doing so, of course, it should avoid interfering with any computation carried on by the current User Mode process. Therefore:

- Before using the coprocessor, the kernel must invoke `kernel_fpu_begin( )`, which essentially calls `save_init_fpu( )` to save the contents of the registers if the User Mode process used the FPU (`TS_USEDFPU` flag), and then resets the TS flag of the `cr0` register.
- After using the coprocessor, the kernel must invoke `kernel_fpu_end( )`, which sets the TS flag of the `cr0` register.

Later, when the User Mode process executes a coprocessor instruction, the `math_state_restore( )` function will restore the contents of the registers, just as in process switch handling.

It should be noted, however, that the execution time of `kernel_fpu_begin( )` is rather large when the current User Mode process is using the coprocessor, so much as to nullify the speedup obtained by using the FPU, MMX, or SSE/SSE2 units. As a matter of fact, the kernel uses them only in a few places, typically when moving or clearing large memory areas or when computing checksum functions.

3.4. Creating Processes

Unix operating systems rely heavily on process creation to satisfy user requests. For example, the shell creates a new process that executes another copy of the shell whenever the user enters a command.

Traditional Unix systems treat all processes in the same way: resources owned by the parent process are duplicated in the child process. This approach makes process creation very slow and inefficient, because it requires copying the entire address space of the parent process. The child process rarely needs to read or modify all the resources inherited from the parent; in many cases, it issues an immediate `execve( )` and wipes out the address space that was so carefully copied.

Modern Unix kernels solve this problem by introducing three different mechanisms:

- The Copy On Write technique allows both the parent and the child to read the same physical pages. Whenever either one tries to write on a physical page, the kernel copies its contents into a new physical page that is assigned to the writing process. The implementation of this technique in Linux is fully explained in [Chapter 9](#).
- Lightweight processes allow both the parent and the child to share many per-process kernel data structures, such as the paging tables (and therefore the entire User Mode address space), the open file tables, and the signal dispositions.
- The `vfork( )` system call creates a process that shares the memory address space of its parent. To prevent the parent from overwriting data needed by the child, the parent's execution is blocked until the child exits or executes a new program. We'll learn more about the `vfork( )` system call in the following section.

3.4.1. The `clone( )`, `fork( )`, and `vfork( )` System Calls

Lightweight processes are created in Linux by using a function named `clone( )`, which uses the following parameters:

`fn`

Specifies a function to be executed by the new process; when the function returns, the child terminates. The function returns an integer, which represents the exit code for the child process.

arg

Points to data passed to the `fn( )` function.

flags

Miscellaneous information. The low byte specifies the signal number to be sent to the parent process when the child terminates; the `SIGCHLD` signal is generally selected. The remaining three bytes encode a group of clone flags, which are shown in [Table 3-8](#).

child_stack

Specifies the User Mode stack pointer to be assigned to the `esp` register of the child process. The invoking process (the parent) should always allocate a new stack for the child.

tls

Specifies the address of a data structure that defines a Thread Local Storage segment for the new lightweight process (see the section "[The Linux GDT](#)" in [Chapter 2](#)). Meaningful only if the `CLONE_SETTLS` flag is set.

ptid

Specifies the address of a User Mode variable of the parent process that will hold the PID of the new lightweight process. Meaningful only if the `CLONE_PARENT_SETTID` flag is set.

ctid

Specifies the address of a User Mode variable of the new lightweight process that will hold the PID of such process. Meaningful only if the `CLONE_CHILD_SETTID` flag is set.

Table 3-8. Clone flags

Flag name	Description
<code>CLONE_VM</code>	Shares the memory descriptor and all Page Tables (see Chapter 9).
<code>CLONE_FS</code>	Shares the table that identifies the root directory and the current working directory, as well as the value of the bitmask used to mask the initial file permissions of a new file (the so-called file umask).
<code>CLONE_FILES</code>	Shares the table that identifies the open files (see Chapter 12).
<code>CLONE_SIGHAND</code>	Shares the tables that identify the signal handlers and the blocked and pending signals (see Chapter 11). If this flag is true, the <code>CLONE_VM</code> flag must also be

CLONE_PTRACE

CLONE_VFORK

CLONE_PARENT

CLONE_THREAD

CLONE_NEWNS

CLONE_SYSVSEM

CLONE_SETTLS

CLONE_PARENT_SETTID

CLONE_CHILD_CLEARTID

CLONE_DETACHED

CLONE_UNTRACED

CLONE_CHILD_SETTID

CLONE_STOPPED

set.

If traced, the parent wants the child to be traced too. Furthermore, the debugger may want to trace the child on its own; in this case, the kernel forces the flag to 1.

Set when the system call issued is a `vfork( )` (see later in this section).

Sets the parent of the child (`parent` and `real_parent` fields in the process descriptor) to the parent of the calling process.

Inserts the child into the same thread group of the parent, and forces the child to share the signal descriptor of the parent. The child's `tgid` and `group_leader` fields are set accordingly. If this flag is true, the `CLONE_SIGHAND` flag must also be set. Set if the clone needs its own namespace, that is, its own view of the mounted filesystems (see [Chapter 12](#)); it is not possible to specify both `CLONE_NEWNS` and `CLONE_FS`.

Shares the System V IPC undoable semaphore operations (see the section "[IPC Semaphores](#)" in [Chapter 19](#)).

Creates a new Thread Local Storage (TLS) segment for the lightweight process; the segment is described in the structure pointed to by the `tls` parameter.

Writes the PID of the child into the User Mode variable of the parent pointed to by the `ptid` parameter.

When set, the kernel sets up a mechanism to be triggered when the child process will exit or when it will start executing a new program. In these cases, the kernel will clear the User Mode variable pointed to by the `ctid` parameter and will awaken any process waiting for this event.

A legacy flag ignored by the kernel.

Set by the kernel to override the value of the `CLONE_PTRACE` flag (used for disabling tracing of kernel threads ; see the section "[Kernel Threads](#)" later in this chapter).

Writes the PID of the child into the User Mode variable of the child pointed to by the `ctid` parameter.

Forces the child to start in the `TASK_STOPPED` state.

`clone( )` is actually a wrapper function defined in the C library (see the section "[POSIX APIs and System Calls](#)" in [Chapter 10](#)), which sets up the stack of the new lightweight process and invokes a `clone( )` system call hidden to the programmer. The `sys_clone( )` service routine that implements the `clone( )` system call does not have the `fn` and `arg` parameters. In fact, the wrapper function saves the pointer `fn` into the child's stack position corresponding to the return address of the wrapper function itself; the pointer `arg` is saved on the child's stack right below `fn`. When the wrapper function terminates, the CPU fetches the return address from the stack and executes the `fn(arg)` function.

The traditional `fork( )` system call is implemented by Linux as a `clone( )` system call whose `flags` parameter specifies both a `SIGCHLD` signal and all the clone flags cleared, and whose `child_stack` parameter is the current parent stack pointer. Therefore, the parent and child temporarily share the same User Mode stack. But thanks to the Copy On Write mechanism, they usually get separate copies of the User Mode stack as soon as one tries to change the stack.

The `vfork( )` system call, introduced in the previous section, is implemented by Linux as a `clone( )` system call whose `flags` parameter specifies both a `SIGCHLD` signal and the flags `CLONE_VM` and `CLONE_VFORK`, and whose `child_stack` parameter is equal to the current parent stack pointer.

3.4.1.1. The `do_fork( )` function

The `do_fork( )` function, which handles the `clone( )`, `fork( )`, and `vfork( )` system calls, acts on the following parameters:

`clone_flags`

Same as the `flags` parameter of `clone( )`

`stack_start`

Same as the `child_stack` parameter of `clone( )`

`regs`

Pointer to the values of the general purpose registers saved into the Kernel Mode stack when switching from User Mode to Kernel Mode (see the section "[The `do\_IRQ\( \)` function](#)" in [Chapter 4](#))

`stack_size`

Unused (always set to 0)

`parent_tidptr`, `child_tidptr`

Same as the corresponding `ptid` and `ctid` parameters of `clone( )`

`do_fork( )` makes use of an auxiliary function called `copy_process( )` to set up the process descriptor and any other kernel data structure required for child's execution. Here are the main steps performed by `do_fork( )`:

1. Allocates a new PID for the child by looking in the `pidmap_array` bitmap (see the earlier section "[Identifying a Process](#)").

2. Checks the `ptrace` field of the parent (`current->ptrace`): if it is not zero, the parent process is being traced by another process, thus `do_fork( )` checks whether the debugger wants to trace the child on its own (independently of the value of the `CLONE_PTRACE` flag specified by the parent); in this case, if the child is not a kernel thread (`CLONE_UNTRACED` flag cleared), the function sets the `CLONE_PTRACE` flag.
3. Invokes `copy_process( )` to make a copy of the process descriptor. If all needed resources are available, this function returns the address of the `task_struct` descriptor just created. This is the workhorse of the forking procedure, and we will describe it right after `do_fork( )`.
4. If either the `CLONE_STOPPED` flag is set or the child process must be traced, that is, the `PT_PTRACED` flag is set in `p->ptrace`, it sets the state of the child to `TASK_STOPPED` and adds a pending `SIGSTOP` signal to it (see the section "[The Role of Signals](#)" in [Chapter 11](#)). The state of the child will remain `TASK_STOPPED` until another process (presumably the tracing process or the parent) will revert its state to `TASK_RUNNING`, usually by means of a `SIGCONT` signal.
5. If the `CLONE_STOPPED` flag is not set, it invokes the `wake_up_new_task( )` function, which performs the following operations:
 - a. Adjusts the scheduling parameters of both the parent and the child (see "[The Scheduling Algorithm](#)" in [Chapter 7](#)).
 - b. If the child will run on the same CPU as the parent,^[*] and parent and child do not share the same set of page tables (`CLONE_VM` flag cleared), it then forces the child to run before the parent by inserting it into the parent's runqueue right before the parent. This simple step yields better performance if the child flushes its address space and executes a new program right after the forking. If we let the parent run first, the Copy On Write mechanism would give rise to a series of unnecessary page duplications.

[*] The parent process might be moved on to another CPU while the kernel forks the new process.
 - c. Otherwise, if the child will not be run on the same CPU as the parent, or if parent and child share the same set of page tables (`CLONE_VM` flag set), it inserts the child in the last position of the parent's runqueue.
6. If the `CLONE_STOPPED` flag is set, it puts the child in the `TASK_STOPPED` state.
7. If the parent process is being traced, it stores the PID of the child in the `ptrace_message` field of `current` and invokes `ptrace_notify( )`, which essentially stops the current process and sends a `SIGCHLD` signal to its parent. The "grandparent" of the child is the debugger that is tracing the parent; the `SIGCHLD` signal notifies the debugger that `current` has forked a child, whose PID can be retrieved by looking into the `current->ptrace_message` field.
8. If the `CLONE_VFORK` flag is specified, it inserts the parent process in a wait queue and suspends it until the child releases its memory address space (that is, until the child either terminates or executes a new program).
9. Terminates by returning the PID of the child.

3.4.1.2. The `copy_process( )` function

The `copy_process( )` function sets up the process descriptor and any other kernel data structure required for a child's execution. Its parameters are the same as `do_fork( )`, plus the PID of the child. Here is a description of its most significant steps:

1. Checks whether the flags passed in the `clone_flags` parameter are compatible. In particular, it returns an error code in the following cases:
 - a. Both the flags `CLONE_NEWNS` and `CLONE_FS` are set.

- b. The `CLONE_THREAD` flag is set, but the `CLONE_SIGHAND` flag is cleared (lightweight processes in the same thread group must share signals).
 - c. The `CLONE_SIGHAND` flag is set, but the `CLONE_VM` flag is cleared (lightweight processes sharing the signal handlers must also share the memory descriptor).
- 2. Performs any additional security checks by invoking `security_task_create( )` and, later, `security_task_alloc( )`. The Linux kernel 2.6 offers hooks for security extensions that enforce a security model stronger than the one adopted by traditional Unix. See [Chapter 20](#) for details.
- 3. Invokes `dup_task_struct( )` to get the process descriptor for the child. This function performs the following actions:
 - a. Invokes `__unlazy_fpu( )` on the current process to save, if necessary, the contents of the FPU, MMX, and SSE/SSE2 registers in the `thread_info` structure of the parent. Later, `dup_task_struct( )` will copy these values in the `thread_info` structure of the child.
 - b. Executes the `alloc_task_struct( )` macro to get a process descriptor (`task_struct` structure) for the new process, and stores its address in the `tsk` local variable.
 - c. Executes the `alloc_thread_info` macro to get a free memory area to store the `thread_info` structure and the Kernel Mode stack of the new process, and saves its address in the `ti` local variable. As explained in the earlier section "[Identifying a Process](#)," the size of this memory area is either 8 KB or 4 KB.
 - d. Copies the contents of the current's process descriptor into the `task_struct` structure pointed to by `tsk`, then sets `tsk->thread_info` to `ti`.
 - e. Copies the contents of the current's `thread_info` descriptor into the structure pointed to by `ti`, then sets `ti->task` to `tsk`.
 - f. Sets the usage counter of the new process descriptor (`tsk->usage`) to 2 to specify that the process descriptor is in use and that the corresponding process is alive (its state is not `EXIT_ZOMBIE` or `EXIT_DEAD`).
 - g. Returns the process descriptor pointer of the new process (`tsk`).
- 4. Checks whether the value stored in `current->signal->rlim[RLIMIT_NPROC].rlim_cur` is smaller than or equal to the current number of processes owned by the user. If so, an error code is returned, unless the process has root privileges. The function gets the current number of processes owned by the user from a per-user data structure named `user_struct`. This data structure can be found through a pointer in the `user` field of the process descriptor.
- 5. Increases the usage counter of the `user_struct` structure (`tsk->user->_count` field) and the counter of the processes owned by the user (`tsk->user->processes`).
- 6. Checks that the number of processes in the system (stored in the `nr_threads` variable) does not exceed the value of the `max_threads` variable. The default value of this variable depends on the amount of RAM in the system. The general rule is that the space taken by all `thread_info` descriptors and Kernel Mode stacks cannot exceed 1/8 of the physical memory. However, the system administrator may change this value by writing in the `/proc/sys/kernel/threads-max` file.
- 7. If the kernel functions implementing the execution domain and the executable format (see [Chapter 20](#)) of the new process are included in kernel modules, it increases their usage counters (see Appendix B).
- 8. Sets a few crucial fields related to the process state:
 - a. Initializes the big kernel lock counter `tsk->lock_depth` to -1 (see the section "[The Big Kernel Lock](#)" in [Chapter 5](#)).
 - b. Initializes the `tsk->did_exec` field to 0: it counts the number of `execve( )` system calls issued by the process.
 - c. Updates some of the flags included in the `tsk->flags` field that have been copied from the parent process: first clears the `PF_SUPERPRIV` flag, which indicates whether the process has used any of its superuser privileges, then sets the `PF_FORKNOEXEC` flag, which indicates that the child has not yet issued an `execve( )` system call.
- 9. Stores the PID of the new process in the `tsk->pid` field.

10. If the `CLONE_PARENT_SETTID` flag in the `clone_flags` parameter is set, it copies the child's PID into the User Mode variable addressed by the `parent_tidptr` parameter.
11. Initializes the `list_head` data structures and the spin locks included in the child's process descriptor, and sets up several other fields related to pending signals, timers, and time statistics.
12. Invokes `copy_semundo( )`, `copy_files( )`, `copy_fs( )`, `copy_sighand( )`, `copy_signal( )`, `copy_mm( )`, and `copy_namespace( )` to create new data structures and copy into them the values of the corresponding parent process data structures, unless specified differently by the `clone_flags` parameter.
13. Invokes `copy_thread( )` to initialize the Kernel Mode stack of the child process with the values contained in the CPU registers when the `clone( )` system call was issued (these values have been saved in the Kernel Mode stack of the parent, as described in [Chapter 10](#)). However, the function forces the value 0 into the field corresponding to the `eax` register (this is the child's return value of the `fork( )` or `clone( )` system call). The `thread.esp` field in the descriptor of the child process is initialized with the base address of the child's Kernel Mode stack, and the address of an assembly language function (`ret_from_fork( )`) is stored in the `thread.eip` field. If the parent process makes use of an I/O Permission Bitmap, the child gets a copy of such bitmap. Finally, if the `CLONE_SETTSL` flag is set, the child gets the TLS segment specified by the User Mode data structure pointed to by the `tls` parameter of the `clone( )` system call.^[*]

[*] A careful reader might wonder how `copy_thread( )` gets the value of the `tls` parameter of `clone( )`, because `tls` is not passed to `do_fork( )` and nested functions. As we'll see in [Chapter 10](#), the parameters of the system calls are usually passed to the kernel by copying their values into some CPU register; thus, these values are saved in the Kernel Mode stack together with the other registers. The `copy_thread( )` function just looks at the address saved in the Kernel Mode stack location corresponding to the value of `esi`.

14. If either `CLONE_CHILD_SETTID` or `CLONE_CHILD_CLEARTID` is set in the `clone_flags` parameter, it copies the value of the `child_tidptr` parameter in the `tsk->set_child_tid` or `tsk->clear_child_tid` field, respectively. These flags specify that the value of the variable pointed to by `child_tidptr` in the User Mode address space of the child has to be changed, although the actual write operations will be done later.
15. Turns off the `TIF_SYSCALL_TRACE` flag in the `thread_info` structure of the child, so that the `ret_from_fork( )` function will not notify the debugging process about the system call termination (see the section "[Entering and Exiting a System Call](#)" in [Chapter 10](#)). (The system call tracing of the child is not disabled, because it is controlled by the `PTRACE_SYSCALL` flag in `tsk->ptrace`.)
16. Initializes the `tsk->exit_signal` field with the signal number encoded in the low bits of the `clone_flags` parameter, unless the `CLONE_THREAD` flag is set, in which case initializes the field to -1. As we'll see in the section "[Process Termination](#)" later in this chapter, only the death of the last member of a thread group (usually, the thread group leader) causes a signal notifying the parent of the thread group leader.
17. Invokes `sched_fork( )` to complete the initialization of the scheduler data structure of the new process. The function also sets the state of the new process to `TASK_RUNNING` and sets the `preempt_count` field of the `thread_info` structure to 1, thus disabling kernel preemption (see the section "[Kernel Preemption](#)" in [Chapter 5](#)). Moreover, in order to keep process scheduling fair, the function shares the remaining timeslice of the parent between the parent and the child (see "[The scheduler_tick\(\) Function](#)" in [Chapter 7](#)).
18. Sets the `cpu` field in the `thread_info` structure of the new process to the number of the local CPU returned by `smp_processor_id( )`.
19. Initializes the fields that specify the parenthood relationships. In particular, if `CLONE_PARENT` or `CLONE_THREAD` are set, it initializes `tsk->real_parent` and `tsk->parent` to the value in `current->real_parent`; the parent of the child thus appears as the parent of the current process. Otherwise, it sets the same fields to `current`.
20. If the child does not need to be traced (`CLONE_PTRACE` flag not set), it sets the `tsk->ptrace`

- field to 0. This field stores a few flags used when a process is being traced by another process. In such a way, even if the current process is being traced, the child will not.
21. Executes the `SET_LINKS` macro to insert the new process descriptor in the process list.
 22. If the child must be traced (`PT_PTRACED` flag in the `tsk->ptrace` field set), it sets `tsk->parent` to `current->parent` and inserts the child into the trace list of the debugger.
 23. Invokes `attach_pid( )` to insert the PID of the new process descriptor in the `pidhash[PIDTYPE_PID]` hash table.
 24. If the child is a thread group leader (flag `CLONE_THREAD` cleared):
 - a. Initializes `tsk->tgid` to `tsk->pid`.
 - b. Initializes `tsk->group_leader` to `tsk`.
 - c. Invokes three times `attach_pid( )` to insert the child in the PID hash tables of type `PIDTYPE_TGID`, `PIDTYPE_PGID`, and `PIDTYPE_SID`.
 25. Otherwise, if the child belongs to the thread group of its parent (`CLONE_THREAD` flag set):
 - a. Initializes `tsk->tgid` to `tsk->current->tgid`.
 - b. Initializes `tsk->group_leader` to the value in `current->group_leader`.
 - c. Invokes `attach_pid( )` to insert the child in the `PIDTYPE_TGID` hash table (more specifically, in the per-PID list of the `current->group_leader` process).
 26. A new process has now been added to the set of processes: increases the value of the `nr_threads` variable.
 27. Increases the `total_forks` variable to keep track of the number of forked processes.
 28. Terminates by returning the child's process descriptor pointer (`tsk`).

Let's go back to what happens after `do_fork( )` terminates. Now we have a complete child process in the runnable state. But it isn't actually running. It is up to the scheduler to decide when to give the CPU to this child. At some future process switch, the schedule bestows this favor on the child process by loading a few CPU registers with the values of the `thread` field of the child's process descriptor. In particular, `esp` is loaded with `thread.esp` (that is, with the address of child's Kernel Mode stack), and `eip` is loaded with the address of `ret_from_fork( )`. This assembly language function invokes the `schedule_tail( )` function (which in turn invokes the `finish_task_switch( )` function to complete the process switch; see the section "[The schedule\(\) Function](#)" in [Chapter 7](#)), reloads all other registers with the values stored in the stack, and forces the CPU back to User Mode. The new process then starts its execution right at the end of the `fork( )`, `vfork( )`, or `clone( )` system call. The value returned by the system call is contained in `eax`: the value is 0 for the child and equal to the PID for the child's parent. To understand how this is done, look back at what `copy_thread( )` does on the `eax` register of the child's process (step 13 of `copy_process( )`).

The child process executes the same code as the parent, except that the fork returns a 0 (see step 13 of `copy_process( )`). The developer of the application can exploit this fact, in a manner familiar to Unix programmers, by inserting a conditional statement in the program based on the PID value that forces the child to behave differently from the parent process.

3.4.2. Kernel Threads

Traditional Unix systems delegate some critical tasks to intermittently running processes, including flushing disk caches, swapping out unused pages, servicing network connections, and so on. Indeed, it is not efficient to perform these tasks in strict linear fashion; both their functions and the end user processes get better response if they are scheduled in the background. Because some of the system processes run only in Kernel Mode, modern operating systems delegate their functions to kernel threads, which are not encumbered with the unnecessary User Mode context. In Linux, kernel threads differ from regular processes in the following ways:

- Kernel threads run only in Kernel Mode, while regular processes run alternatively in Kernel Mode and in User Mode.
- Because kernel threads run only in Kernel Mode, they use only linear addresses greater than `PAGE_OFFSET`. Regular processes, on the other hand, use all four gigabytes of linear addresses, in either User Mode or Kernel Mode.

3.4.2.1. Creating a kernel thread

The `kernel_thread( )` function creates a new kernel thread. It receives as parameters the address of the kernel function to be executed (`fn`), the argument to be passed to that function (`arg`), and a set of clone flags (`flags`). The function essentially invokes `do_fork( )` as follows:

```
do_fork(flags|CLONE_VM|CLONE_UNTRACED, 0, pregs, 0, NULL, NULL);
```

The `CLONE_VM` flag avoids the duplication of the page tables of the calling process: this duplication would be a waste of time and memory, because the new kernel thread will not access the User Mode address space anyway. The `CLONE_UNTRACED` flag ensures that no process will be able to trace the new kernel thread, even if the calling process is being traced.

The `pregs` parameter passed to `do_fork( )` corresponds to the address in the Kernel Mode stack where the `copy_thread( )` function will find the initial values of the CPU registers for the new thread. The `kernel_thread( )` function builds up this stack area so that:

- The `ebx` and `edx` registers will be set by `copy_thread( )` to the values of the parameters `fn` and `arg`, respectively.
- The `eip` register will be set to the address of the following assembly language fragment:

```
movl %edx,%eax
pushl %edx
call *%ebx
pushl %eax
call do_exit
```

Therefore, the new kernel thread starts by executing the `fn(arg)` function. If this function terminates, the kernel thread executes the `_exit( )` system call passing to it the return value of `fn( )` (see the section "[Destroying Processes](#)" later in this chapter).

3.4.2.2. Process 0

The ancestor of all processes, called process 0, the idle process, or, for historical reasons, the swapper process, is a kernel thread created from scratch during the initialization phase of Linux (see Appendix A). This ancestor process uses the following statically allocated data structures (data structures for all other processes are dynamically allocated):

- A process descriptor stored in the `init_task` variable, which is initialized by the `INIT_TASK` macro.
- A `thread_info` descriptor and a Kernel Mode stack stored in the `init_thread_union` variable and initialized by the `INIT_THREAD_INFO` macro.

- The following tables, which the process descriptor points to:

- ◆ `init_mm`
- ◆ `init_fs`
- ◆ `init_files`
- ◆ `init_signals`
- ◆ `init_sighand`

The tables are initialized, respectively, by the following macros:

- ◆ `INIT_MM`
- ◆ `INIT_FS`
- ◆ `INIT_FILES`
- ◆ `INIT_SIGNALS`
- ◆ `INIT_SIGHAND`

- The master kernel Page Global Directory stored in `swapper_pg_dir` (see the section "[Kernel Page Tables](#)" in [Chapter 2](#)).

The `start_kernel( )` function initializes all the data structures needed by the kernel, enables interrupts, and creates another kernel thread, named process 1 (more commonly referred to as the init process):

```
kernel_thread(init, NULL, CLONE_FS|CLONE_SIGHAND);
```

The newly created kernel thread has PID 1 and shares all per-process kernel data structures with process 0. When selected by the scheduler, the init process starts executing the `init( )` function.

After having created the init process, process 0 executes the `cpu_idle( )` function, which essentially consists of repeatedly executing the `hlt` assembly language instruction with the interrupts enabled (see [Chapter 4](#)). Process 0 is selected by the scheduler only when there are no other processes in the `TASK_RUNNING` state.

In multiprocessor systems there is a process 0 for each CPU. Right after the power-on, the BIOS of the computer starts a single CPU while disabling the others. The swapper process running on CPU 0 initializes the kernel data structures, then enables the other CPUs and creates the additional swapper processes by means of the `copy_process( )` function passing to it the value 0 as the new PID. Moreover, the kernel sets the `cpu` field of the `thread_info` descriptor of each forked process to the proper CPU index.

3.4.2.3. Process 1

The kernel thread created by process 0 executes the `init( )` function, which in turn completes the initialization of the kernel. Then `init( )` invokes the `execve( )` system call to load the executable program `init`. As a result, the init kernel thread becomes a regular process having its own per-process kernel data structure (see [Chapter 20](#)). The init process stays alive until the system is shut down, because it creates and monitors the activity of all processes that implement the outer layers of the operating system.

3.4.2.4. Other kernel threads

Linux uses many other kernel threads. Some of them are created in the initialization phase and run until shutdown; others are created "on demand," when the kernel must execute a task that is better performed in its own execution context.

A few examples of kernel threads (besides process 0 and process 1) are:

keventd (also called events)

Executes the functions in the `keventd_wq` workqueue (see [Chapter 4](#)).

kapmd

Handles the events related to the Advanced Power Management (APM).

kswapd

Reclaims memory, as described in the section "[Periodic Reclaiming](#)" in [Chapter 17](#).

pdflush

Flushes "dirty" buffers to disk to reclaim memory, as described in the section "[The pdflush Kernel Threads](#)" in [Chapter 15](#).

kblockd

Executes the functions in the `kblockd_workqueue` workqueue. Essentially, it periodically activates the block device drivers, as described in the section "[Activating the Block Device Driver](#)" in [Chapter 14](#).

ksoftirqd

Runs the tasklets (see section "[Softirqs and Tasklets](#)" in [Chapter 4](#)); there is one of these kernel threads for each CPU in the system.

3.5. Destroying Processes

Most processes "die" in the sense that they terminate the execution of the code they were supposed to run. When this occurs, the kernel must be notified so that it can release the resources owned by the process; this includes memory, open files, and any other odds and ends that we will encounter in this book, such as semaphores.

The usual way for a process to terminate is to invoke the `exit( )` library function, which releases the resources allocated by the C library, executes each function registered by the programmer, and ends up invoking a system call that evicts the process from the system. The `exit( )` library function may be inserted by the programmer explicitly. Additionally, the C compiler always inserts an `exit( )` function call right after the last statement of the `main( )` function.

Alternatively, the kernel may force a whole thread group to die. This typically occurs when a process in the group has received a signal that it cannot handle or ignore (see [Chapter 11](#)) or when an unrecoverable CPU exception has been raised in Kernel Mode while the kernel was running on behalf of the process (see [Chapter 4](#)).

3.5.1. Process Termination

In Linux 2.6 there are two system calls that terminate a User Mode application:

- The `exit_group( )` system call, which terminates a full thread group, that is, a whole multithreaded application. The main kernel function that implements this system call is called `do_group_exit( )`. This is the system call that should be invoked by the `exit( )` C library function.
- The `_exit( )` system call, which terminates a single process, regardless of any other process in the thread group of the victim. The main kernel function that implements this system call is called `do_exit( )`. This is the system call invoked, for instance, by the `pthread_exit( )` function of the LinuxThreads library.

3.5.1.1. The `do_group_exit( )` function

The `do_group_exit( )` function kills all processes belonging to the thread group of `current`. It receives as a parameter the process termination code, which is either a value specified in the `exit_group( )` system call (normal termination) or an error code supplied by the kernel (abnormal termination). The function executes the following operations:

1. Checks whether the `SIGNAL_GROUP_EXIT` flag of the exiting process is not zero, which means that the kernel already started an exit procedure for this thread group. In this case, it considers as exit code the value stored in `current->signal->group_exit_code`, and jumps to step 4.
2. Otherwise, it sets the `SIGNAL_GROUP_EXIT` flag of the process and stores the termination code in the `current->signal->group_exit_code` field.
3. Invokes the `zap_other_threads( )` function to kill the other processes in the thread group of `current`, if any. In order to do this, the function scans the per-PID list in the `PIDTYPE_TGID` hash table corresponding to `current->tgid`; for each process in the list different from `current`, it

sends a SIGKILL signal to it (see [Chapter 11](#)). As a result, all such processes will eventually execute the `do_exit( )` function, and thus they will be killed.

4. Invokes the `do_exit( )` function passing to it the process termination code. As we'll see below, `do_exit( )` kills the process and never returns.

3.5.1.2. The `do_exit( )` function

All process terminations are handled by the `do_exit( )` function, which removes most references to the terminating process from kernel data structures. The `do_exit( )` function receives as a parameter the process termination code and essentially executes the following actions:

1. Sets the `PF_EXITING` flag in the `flag` field of the process descriptor to indicate that the process is being eliminated.
2. Removes, if necessary, the process descriptor from a dynamic timer queue via the `del_timer_sync( )` function (see [Chapter 6](#)).
3. Detaches from the process descriptor the data structures related to paging, semaphores, filesystem, open file descriptors, namespaces, and I/O Permission Bitmap, respectively, with the `exit_mm( )`, `exit_sem( )`, `_exit_files( )`, `_exit_fs( )`, `exit_namespace( )`, and `exit_thread( )` functions. These functions also remove each of these data structures if no other processes are sharing them.
4. If the kernel functions implementing the execution domain and the executable format (see [Chapter 20](#)) of the process being killed are included in kernel modules, the function decreases their usage counters.
5. Sets the `exit_code` field of the process descriptor to the process termination code. This value is either the `_exit( )` or `exit_group( )` system call parameter (normal termination), or an error code supplied by the kernel (abnormal termination).
6. Invokes the `exit_notify( )` function to perform the following operations:
 - a. Updates the parenthood relationships of both the parent process and the child processes. All child processes created by the terminating process become children of another process in the same thread group, if any is running, or otherwise of the init process.
 - b. Checks whether the `exit_signal` process descriptor field of the process being terminated is different from `-1`, and whether the process is the last member of its thread group (notice that these conditions always hold for any normal process; see step 16 in the description of `copy_process( )` in the earlier section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)"). In this case, the function sends a signal (usually `SIGCHLD`) to the parent of the process being terminated to notify the parent about a child's death.
 - c. Otherwise, if the `exit_signal` field is equal to `-1` or the thread group includes other processes, the function sends a `SIGCHLD` signal to the parent only if the process is being traced (in this case the parent is the debugger, which is thus informed of the death of the lightweight process).
 - d. If the `exit_signal` process descriptor field is equal to `-1` and the process is not being traced, it sets the `exit_state` field of the process descriptor to `EXIT_DEAD`, and invokes `release_task( )` to reclaim the memory of the remaining process data structures and to decrease the usage counter of the process descriptor (see the following section). The usage counter becomes equal to 1 (see step 3f in the `copy_process( )` function), so that the process descriptor itself is not released right away.
 - e. Otherwise, if the `exit_signal` process descriptor field is not equal to `-1` or the process is being traced, it sets the `exit_state` field to `EXIT_ZOMBIE`. We'll see what happens to zombie processes in the following section.
 - f. Sets the `PF_DEAD` flag in the `flags` field of the process descriptor (see the section "[The schedule\(\) Function](#)" in [Chapter 7](#)).

7. Invokes the `schedule( )` function (see [Chapter 7](#)) to select a new process to run. Because a process in an `EXIT_ZOMBIE` state is ignored by the scheduler, the process stops executing right after the `switch_to` macro in `schedule( )` is invoked. As we'll see in [Chapter 7](#), the scheduler will check the `PF_DEAD` flag and will decrease the usage counter in the descriptor of the zombie process being replaced to denote the fact that the process is no longer alive.

3.5.2. Process Removal

The Unix operating system allows a process to query the kernel to obtain the PID of its parent process or the execution state of any of its children. A process may, for instance, create a child process to perform a specific task and then invoke some `wait( )`-like library function to check whether the child has terminated. If the child has terminated, its termination code will tell the parent process if the task has been carried out successfully.

To comply with these design choices, Unix kernels are not allowed to discard data included in a process descriptor field right after the process terminates. They are allowed to do so only after the parent process has issued a `wait( )`-like system call that refers to the terminated process. This is why the `EXIT_ZOMBIE` state has been introduced: although the process is technically dead, its descriptor must be saved until the parent process is notified.

What happens if parent processes terminate before their children? In such a case, the system could be flooded with zombie processes whose process descriptors would stay forever in RAM. As mentioned earlier, this problem is solved by forcing all orphan processes to become children of the `init` process. In this way, the `init` process will destroy the zombies while checking for the termination of one of its legitimate children through a `wait( )`-like system call.

The `release_task( )` function detaches the last data structures from the descriptor of a zombie process; it is applied on a zombie process in two possible ways: by the `do_exit( )` function if the parent is not interested in receiving signals from the child, or by the `wait4( )` or `waitpid( )` system calls after a signal has been sent to the parent. In the latter case, the function also will reclaim the memory used by the process descriptor, while in the former case the memory reclaiming will be done by the scheduler (see [Chapter 7](#)). This function executes the following steps:

1. Decreases the number of processes belonging to the user owner of the terminated process. This value is stored in the `user_struct` structure mentioned earlier in the chapter (see step 4 of `copy_process( )`).
2. If the process is being traced, the function removes it from the debugger's `ptrace_children` list and assigns the process back to its original parent.
3. Invokes `_exit_signal( )` to cancel any pending signal and to release the `signal_struct` descriptor of the process. If the descriptor is no longer used by other lightweight processes, the function also removes this data structure. Moreover, the function invokes `exit_itimers( )` to detach any POSIX interval timer from the process.
4. Invokes `_exit_sighand( )` to get rid of the signal handlers.
5. Invokes `_unhash_process( )`, which in turn:
 - a. Decreases by 1 the `nr_threads` variable.
 - b. Invokes `detach_pid( )` twice to remove the process descriptor from the `pidhash` hash tables of type `PIDTYPE_PID` and `PIDTYPE_TGID`.
 - c. If the process is a thread group leader, invokes again `detach_pid( )` twice to remove the process descriptor from the `PIDTYPE_PGID` and `PIDTYPE_SID` hash tables.
 - d. Uses the `REMOVE_LINKS` macro to unlink the process descriptor from the process list.

6. If the process is not a thread group leader, the leader is a zombie, and the process is the last member of the thread group, the function sends a signal to the parent of the leader to notify it of the death of the process.
7. Invokes the `sched_exit ( )` function to adjust the timeslice of the parent process (this step logically complements step 17 in the description of `copy_process ( )`)
8. Invokes `put_task_struct ( )` to decrease the process descriptor's usage counter; if the counter becomes zero, the function drops any remaining reference to the process:
 - a. Decreases the usage counter (`_count` field) of the `user_struct` data structure of the user that owns the process (see step 5 of `copy_process ( )`), and releases that data structure if the usage counter becomes zero.
 - b. Releases the process descriptor and the memory area used to contain the `thread_info` descriptor and the Kernel Mode stack.